INFRASTRUCTURE & DEVOPS

MediBot

AI-Powered Patient Triage System

Complete Project Documentation

- Part I — Infrastructure Setup & Teardown Guide
- Part II — Kubernetes Manifests Explained
- Part III — Jenkinsfile Explained

Terraform • Amazon EKS • Jenkins • SonarQube • Nexus
Docker • Amazon ECR • ArgoCD (GitOps) • Prometheus & Grafana • AWS Bedrock

Generated October 29, 2026

Table of Contents

Complete Infrastructure Setup & Teardown Guide . . . 8
1. Architecture Overview . . . 8
Component Inventory . . . 8
2. How the Pieces Talk to Each Other . . . 9
3. Secrets & Credentials Master Reference . . . 11
4. Fixed vs. Changing Values . . . 13
5. Prerequisites . . . 13
6. Known Architecture Quirks & Things to Decide On . . . 14
6.1 — Terraform provisions a real RDS instance the app never uses . . . 14
6.2 — The sample app Deployment points at someone else's ECR image . . . 15
6.3 — mysql-secret's password ships as a visible sample value . . . 15
Phase 1 — GitHub Repos + Token . . . 15
Step 1 — Create GitHub Repos . . . 15
Step 2 — Create a GitHub Personal Access Token . . . 15
Phase 2 — AWS Account Setup . . . 16
Step 3 — Create an IAM User for Terraform (Console Access Only — No Access Keys) . . . 16
Step 4 — Create an EC2 Key Pair . . . 17
Phase 3 — Management Server . . . 17
Step 5 — Create the Management EC2 Instance . . . 17
Step 6 — Install Tools on the Management Server . . . 18
Step 7 — Connect the Management Server to AWS . . . 19
Step 8 — Clone the Terraform Repo and Deploy Infrastructure . . . 20
Phase 4 — EKS Setup . . . 21
Step 9 — Connect kubectl to the New EKS Cluster . . . 21
Step 10 — Create an OIDC Provider for the Cluster . . . 21
Step 11 — Install the AWS Load Balancer Controller . . . 21
Step 12 — Install Metrics Server . . . 23
Step 13 — Verify the EBS CSI Driver (Already Installed by Terraform) . . . 23
Phase 5 — Bedrock IAM Permission . . . 24
Step 14 — Attach the Bedrock Policy to the EKS Node Role . . . 24
Phase 6 — ECR Setup . . . 25
Step 15 — Create the ECR Repository . . . 25

Step 16 — Create an IAM User Dedicated to Jenkins' ECR Push . . . 25
Phase 7 — Jenkins Setup . . . 26
Step 17 — SSH into the Jenkins EC2 . . . 26
Step 18 — Install Java + Jenkins . . . 26
Step 19 — Install Docker on the Jenkins EC2 . . . 26
Step 20 — Install kubectl on the Jenkins EC2 . . . 27
Step 21 — Install Trivy on the Jenkins EC2 . . . 27
Step 22 — Install AWS CLI on the Jenkins EC2 . . . 27
Step 23 — Copy the kubeconfig to the Jenkins User . . . 27
Phase 8 — SonarQube Setup . . . 28
Step 24 — Connect to the SonarQube EC2 via SSM . . . 28
Step 25 — Install SonarQube (as a Docker Container) . . . 28
Step 26 — Configure SonarQube . . . 28
Phase 9 — Nexus Setup . . . 29
Step 27 — Connect to the Nexus EC2 via SSM . . . 29
Step 28 — Install Nexus (as a Docker Container) . . . 29
Step 29 — Configure Nexus . . . 29
Step 30 — Update pom.xml with the Current Nexus URLs . . . 30
Phase 10 — Jenkins Configuration . . . 31
Step 31 — Open Jenkins and Run Initial Setup . . . 31
Step 32 — Install Jenkins Plugins . . . 31
Step 33 — Configure Tools . . . 31
Step 34 — Add All Credentials . . . 32
Step 35 — Configure the SonarQube Server in Jenkins . . . 32
Step 36 — Add Nexus Maven Settings . . . 32
Phase 11 — Kubernetes Namespace . . . 33
Step 37 — Create the Namespace . . . 33
Phase 12 — Review the Manifests Repo . . . 33
Step 38 — The EBS StorageClass . . . 34
Step 39 — The MySQL Manifest . . . 34
Step 40 — The MediBot App Manifest . . . 36
Step 41 — The HPA Manifest . . . 38
Step 42 — The Ingress Manifest . . . 38
Step 43 — Nothing to Commit (Unless You Changed Something) . . . 39
Phase 13 — ArgoCD Setup . . . 39

Step 44 — Install ArgoCD . . . 39
Step 45 — Configure ArgoCD . . . 40
Phase 14 — Jenkins Pipeline Job . . . 41
Step 46 — Create the Pipeline Job . . . 41
Step 47 — Set Up the GitHub Webhook . . . 41
Step 48 — Run the Pipeline for the First Time . . . 42
Phase 15 — Verify . . . 42
Step 49 — Verify the Pods . . . 42
Step 50 — Access the Application . . . 42
Phase 16 — Monitoring . . . 43
Step 51 — Install Prometheus + Grafana . . . 43
Step 52 — Access Grafana . . . 44
Phase 17 — Domain + SSL Setup . . . 44
Step 53 — ACM Certificate . . . 44
Step 54 — Confirm ingress.yaml Still Has the Right Certificate ARN . . . 45
Step 55 — Add the Route53 A Record . . . 45
Step 56 — Test . . . 45
Post-Deployment Verification . . . 46
Troubleshooting . . . 46
IP Address Dependencies . . . 47
Optional — Attach Elastic IPs to Avoid This Permanently . . . 47
Complete Teardown Manual . . . 48
Step 1 — Delete the ArgoCD Application First . . . 48
*Step 2 — Delete Kubernetes-Managed Load Balancers *Before* the Cluster* . . . 48
Step 3 — Let PVCs Release Cleanly (Avoid Orphaned EBS Volumes) . . . 48
Step 4 — Terraform Destroy . . . 48
Step 5 — Check for Leftover EBS Volumes . . . 49
Step 6 — Delete the eksctl-Created CloudFormation Stack(s) . . . 49
Step 7 — Delete the IAM Policies (Only After Step 6) . . . 49
Step 8 — Force-Delete the ECR Repository . . . 49
Step 9 — Delete the IAM User and Its Access Key . . . 50
Step 10 — Delete the EC2 Key Pair . . . 50
Step 11 — Terminate the medibot-admin EC2 . . . 50
Step 12 — Clean Up the Dangling Route53 Record . . . 50
Step 13 — Final Verification Sweep . . . 50

A field-by-field, beginner-friendly walkthrough of everything in medibot-manifests . . . 53

1. Kubernetes Basics, in Plain English . . . 53

2. The Big Picture — 5 Files, 9 Resources . . . 54

3. storageclass.yaml Explained . . . 54

4. mysql.yaml Explained . . . 56

4.1 — The Secret . . . 56

4.2 — The PersistentVolumeClaim . . . 57

4.3 — The Deployment . . . 57

4.4 — The Service . . . 59

5. deployment.yaml Explained . . . 60

5.1 — The ConfigMap . . . 60

5.2 — The Deployment . . . 61

5.3 — The Service . . . 64

6. hpa.yaml Explained . . . 65

7. ingress.yaml Explained . . . 66

Annotations, one at a time . . . 67

spec . . . 67

The full request path, end to end . . . 68

8. The Complete Reference Map . . . 68

9. General Beginner FAQ . . . 70

10. Command Cheat Sheet . . . 71

A stage-by-stage, beginner-friendly walkthrough of the entire CI pipeline . . . 74

1. What a Jenkinsfile Actually Is . . . 74

2. The Full File, For Reference . . . 74

3. The pipeline {} Block's Top-Level Keys . . . 77

4. The environment {} Block, Line by Line . . . 78

5. Every Stage, In Full Detail . . . 79

Stage 1 — Git Checkout . . . 79

Stage 2 — Compile . . . 80

Stage 3 — Test . . . 80

Stage 4 — Trivy FS Scan . . . 80

Stage 5 — SonarQube Analysis . . . 81

Stage 6 — Quality Gate . . . 81

Stage 7 — Build JAR . . . 82

Stage 8 — Publish to Nexus . . . 83

Stage 9 — Docker Build 83
Stage 10 — Trivy Image Scan 84
Stage 11 — Push to ECR 84
Stage 12 — Update Manifests Repo 85
6. Master Credentials Table 87
7. Master Plugin Table 87
8. The Groovy String Quoting Trap — Read This Once 88
9. What Triggers This Pipeline, End to End 89
10. General Beginner FAQ 89

PART I

Infrastructure Setup & Teardown Guide

Terraform → EKS → Jenkins → SonarQube → Nexus → ArgoCD → Prometheus/Grafana → Route53/ACM HTTPS, plus a full battle-tested teardown manual.

Complete Infrastructure Setup & Teardown Guide

> *A full, beginner-friendly walkthrough of standing up a production-style CI/CD + GitOps stack on AWS: Terraform → EKS → Jenkins → SonarQube → Nexus → ECR → ArgoCD → Prometheus/Grafana → Route53/ACM HTTPS. Every step includes **what to run**, **why it's needed**, **what breaks if you skip it**, and **where any secret/credential it produces gets used later**. This revision was re-verified directly against the actual Terraform (`main.tf`) and Kubernetes manifest files used in this project, not just the original walkthrough notes — see Section 6 for things that turned up that weren't obvious from the original guide.*

---

1. Architecture Overview

MediBot is a Spring Boot application (AI patient triage, backed by AWS Bedrock) deployed on Amazon EKS, built and shipped through a classic **CI/CD + GitOps** split: Jenkins does Continuous Integration, ArgoCD does Continuous Deployment, and they never talk to each other directly — they're connected only through a Git repository.

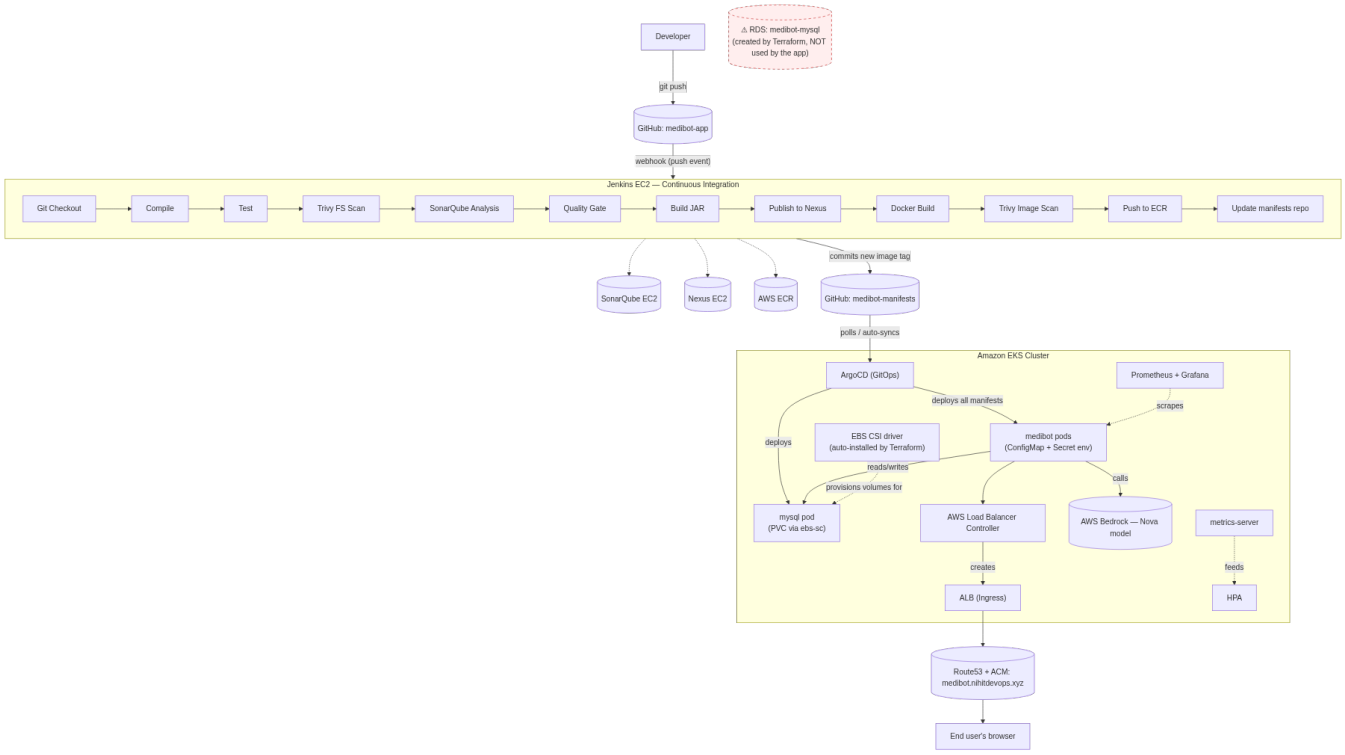


Figure 1.1 — End-to-end CI/CD and GitOps architecture

> Notice the dashed red box: Terraform provisions a real RDS MySQL instance that the application **does not actually connect to** — see Section 6 for why, and what to do about it.

Component Inventory

#	Component	Runs as	Purpose
1	GitHub repos (`medibot-app`, `medibot-infra`, `medibot-manifests`)	—	Source of truth, split source / infra-as-code / CD repos
2	Management server	EC2	Runs Terraform + kubectl safely, outside your laptop
3	Terraform	CLI on management server	Provisions VPC, 3 EC2s, EKS cluster, EBS CSI driver, RDS (unused — see Section 6), IAM roles
4	Jenkins	EC2 (bare install, not containerized)	CI: build, test, scan, publish, push, trigger CD
5	SonarQube	Docker container on its own EC2	Static code analysis + quality gate
6	Nexus	Docker container on its own EC2	Hosts built `.jar` artifacts (Maven repo)
7	AWS ECR	Managed AWS service	Hosts built Docker images
8	AWS Load Balancer Controller	Pod in EKS (`kube-system`)	Turns Kubernetes Ingress objects into real AWS ALBs
9	EBS CSI driver	EKS addon (auto-installed by Terraform)	Lets Kubernetes PersistentVolumeClaims actually provision real EBS volumes
10	metrics-server	Pod in EKS (`kube-system`)	Feeds CPU/memory metrics to the Horizontal Pod Autoscaler
11	ArgoCD	Pods in EKS (`argocd` namespace)	CD: watches `medibot-manifests`, auto-deploys to the cluster
12	MediBot app	Pods in EKS (`webapps` namespace)	The actual application
13	MySQL	Pod in EKS (`webapps` namespace), EBS-backed	The application's database — **in-cluster, not RDS**
14	Prometheus + Grafana	Pods in EKS (`monitoring` namespace)	Metrics collection + dashboards
15	AWS Bedrock	Managed AWS service	The actual AI model used for triage
16	Route53 + ACM	Managed AWS service	DNS + free TLS certificate for HTTPS

---

2. How the Pieces Talk to Each Other

This is the part that's easy to lose track of — here's exactly which credential or network path connects each pair of systems.

From	To	How it connects	Secret/credential used
GitHub (`medibot-app`)	Jenkins	Webhook HTTP POST to `http://<jenkins-ip>:8080/github-webhook/` on every push	none (public endpoint, but only triggers a poll — Jenkins still authenticates the actual `git clone` separately)
Jenkins	GitHub	`git clone`/`git push` over HTTPS using a token	Jenkins credential `git` (username + GitHub PAT from Step 2)
Jenkins	SonarQube	REST API call during `SonarQube Analysis` stage, authenticated with a token	Jenkins credential `sonar-token` (SonarQube token from Step 25)
SonarQube	Jenkins	Webhook callback `http://<jenkins-ip>:8080/sonarqube-webhook/` to report analysis result, which unblocks the `Quality Gate` pipeline stage	none (one-way notification, not authenticated — this is why the Quality Gate stage would hang forever without it)
Jenkins	Nexus	`mvn deploy`, authenticated via Maven's `settings.xml`	Jenkins Managed File `medibot-nexus` (contains the Nexus `jenkins` user password from Step 28)
pom.xml (in `medibot-app`)	Nexus	Declares *where* to publish (URLs + repo IDs) — no password here by design, since this file lives in a public-ish git repo	none directly — the password lives only in Jenkins' `settings.xml`, matched by the `<id>` tag
Jenkins	AWS ECR	`aws ecr get-login-password` then `docker push`, authenticated with IAM access keys	Jenkins credential `aws-creds` (IAM user `medibot-jenkins-ecr` from Step 15)
Jenkins	`medibot-manifests` repo	`git clone` + `sed` to bump the image tag + `git push`, same mechanism as the `git` credential above	Jenkins credential `git`
ArgoCD	`medibot-manifests` repo	Polls over HTTPS using a GitHub token, configured once in the ArgoCD UI	ArgoCD's own stored repo credential (GitHub username + PAT, entered in Step 44 — separate copy of the same token from Step 2)
ArgoCD	Kubernetes API (EKS)	ArgoCD runs *inside* the cluster, so it talks to `https://kubernetes.default.svc` — a DNS name every cluster resolves internally to its own API server	none — this is why the ArgoCD "Cluster URL" field can be the same generic value on every cluster everywhere
MediBot pods	MySQL pod	Plain TCP on port 3306, via the in-cluster Service DNS name `mysql-service`	`mysql-secret` (Kubernetes Secret — root password), injected into both pods' env

From	To	How it connects	Secret/credential used
EKS pods (ALB Controller)	AWS API	IRSA (IAM Roles for Service Accounts) — the pod's Kubernetes ServiceAccount is federated to an IAM Role via the cluster's OIDC provider	IAM Policy `AWSLoadBalancerControllerIAMPolicy`, attached through `eksctl create iamserviceaccount` (Step 11) — no static keys involved at all
EKS nodes (EBS CSI driver)	AWS API	IRSA, set up entirely by Terraform (`ebs_csi_role` + `aws_eks_addon.ebs_csi`)	IAM Policy `AmazonEBSCSIDriverPolicy` — no manual step needed, it's already done by the time you reach Step 9
EKS pods (MediBot app)	AWS Bedrock	The EKS **node role** (`medibot-eks-node-role`) is granted `bedrock:InvokeModel` directly — pods inherit this via the underlying EC2 instance profile	IAM Policy `medibot-bedrock-policy` (Step 14)
Ingress (ALB)	ACM	The Ingress YAML references a certificate ARN directly in an annotation	ACM certificate ARN (Step 53/54)
Route53	ALB	An Alias A record points `medibot.nihitdevops.xyz` at the ALB's DNS name	none — Alias records don't need a static IP, they resolve dynamically

The single most important idea in this whole architecture: Jenkins *never* touches the Kubernetes cluster directly to deploy anything. It stops after pushing a new image tag into the `medibot-manifests` Git repo. ArgoCD is the only thing that ever talks to the cluster for deployment — it notices the Git change and reconciles the cluster to match. This is what "GitOps" means in practice: **Git is the single source of truth for what should be running**, not a `kubectl apply` command buried in a pipeline somewhere.

---

3. Secrets & Credentials Master Reference

Keep this table open while you work — every secret created below maps to exactly one row here, and every row tells you **everywhere** that secret gets used later, so you never have to guess.

Secret / Credential	Created in	Stored as	Used by (where exactly)
GitHub Personal Access Token	Step 2	Jenkins credential `git` (Username+Password kind); also pasted again into ArgoCD's repo connection (Step 44)	Jenkins `Git Checkout` stage, Jenkins `Update Manifests Repo` stage, GitHub webhook auth (implicitly, via repo access), ArgoCD repo polling

Secret / Credential	Created in	Stored as	Used by (where exactly)
`medibot-admin` console password	Step 3	Your memory / password manager — no access key ever exists for this user	Logging into the AWS Console during `aws login --remote` (Step 7), which produces short-lived, auto-refreshing CLI credentials used by `terraform apply`/`destroy` — never touches Jenkins
EC2 key pair `delete_later` (`.pem` file)	Step 4	Your local machine only	SSH into the management server and the Jenkins server (SonarQube/Nexus use SSM instead — no key needed)
Terraform `db_password` variable	Supplied at `terraform apply` time (Step 8)	**Never committed to Git** — pass via `-var`, a gitignored `.tfvars` file, or `TF_VAR_db_password`	Only used by the unused RDS instance (Section 6) — if you delete the RDS resources, this variable can be deleted too
AWS IAM user `medibot-jenkins-ecr` access key	Step 15	Jenkins credential `aws-creds` (AWS Credentials kind)	Jenkins `Push to ECR` stage
AWS Account ID	—	Jenkins credential `aws-account-id` (Secret Text kind)	Jenkinsfile `environment{}` block, to build the ECR registry URL (`<account-id>.dkr.ecr.<region>.amazonaws.com`)
SonarQube admin password	Step 25	Your memory / password manager	Logging into the SonarQube UI only
SonarQube analysis token (`jenkins-token`)	Step 25	Jenkins credential `sonar-token` (Secret Text kind)	Jenkinsfile `SonarQube Analysis` stage AND `Quality Gate` stage; also referenced by name in the Jenkins global "SonarQube servers" config (Step 34)
Nexus admin password	Step 28	Your memory / password manager	Logging into the Nexus UI only
Nexus `jenkins` user password	Step 28	Jenkins Managed File `medibot-nexus` (inside a `settings.xml`)	Jenkinsfile `Publish to Nexus` stage — **must match exactly** what you typed in Nexus's UI, or you get a 401
`mysql-secret` (`MYSQL_ROOT_PASSWORD`, `MYSQL_DATABASE`)	Step 39, inside `mysql.yaml`	A plain Kubernetes `Secret` (base64-encoded, **not encrypted at rest by default**) in the `webapps` namespace	MySQL pod's own startup env, and the `medibot` app pod's `MYSQL_PASSWORD` env
ACM certificate ARN	Step 53/54	Pasted directly into `ingress.yaml`	The Ingress annotation `alb.ingress.kubernetes.io/certificate-arn`
`AWSLoadBalancerControllerIAMPolicy`	Step 11	IAM policy, attached via IRSA — no static credential	The `aws-load-balancer-controller` pod's permissions to create ALBs

Secret / Credential	Created in	Stored as	Used by (where exactly)
`medibot-bedrock-policy`	Step 14	IAM policy attached to the EKS node role — no static credential	MediBot pods calling `bedrock:InvokeModel`

> **About `mysql-secret`:** a plain Kubernetes `Secret` is only base64-*encoded*, not encrypted, and anyone with `kubectl get secret -o yaml` access to the namespace can trivially decode it. That's fine for a learning project, but **never commit a real production password as the literal `stringData` value in a YAML file checked into Git** — anyone with read access to the repo (and its full Git history, even after you change it later) can see it. Treat whatever you put here as already-compromised the moment you commit it, and change it before this ever becomes a real system.

Rule of thumb: anything Jenkins needs lives in **Jenkins Credentials** or a **Jenkins Managed File** — never hardcoded in the Jenkinsfile or committed to Git. Anything the cluster itself needs (ALB controller, EBS CSI driver, Bedrock access) uses **IAM roles**, not static keys, so there's nothing to leak or rotate. The one deliberate exception is `mysql-secret`, which is a plain Kubernetes Secret because standing up a proper secrets manager (AWS Secrets Manager + External Secrets Operator, or Sealed Secrets) is outside the scope of this guide — just be aware of the trade-off you're making.

---

4. Fixed vs. Changing Values

Value	Status	Notes
AWS Account ID	**Fixed**: `719152986284`	Same every redeploy, same AWS account
Region	**Fixed**: `ap-south-1`	
Domain	**Fixed**: `nihitdevops.xyz`	Hosted zone + `*.nihitdevops.xyz` ACM cert are deliberately kept across teardowns — reuse them, don't recreate
VPC ID	**Changes every redeploy**	Pull fresh via `terraform output` or `aws ec2 describe-vpcs`
All 4 EC2 public IPs	**Changes every redeploy** (no Elastic IPs by default)	See IP Address Dependencies
EKS OIDC provider ID	**Changes every redeploy**	Regenerated each time by `eksctl utils associate-iam-oidc-provider`
ALB / NLB DNS names	**Changes every redeploy**	New load balancer, new DNS name, every time

---

5. Prerequisites

- An AWS account with billing enabled, and enough IAM permissions on your own user to create an `AdministratorAccess` IAM user (Step 3)
- A GitHub account

- A registered domain (or subdomain) you can manage DNS for — this guide uses `nihitdevops.xyz`
- A local terminal with SSH and a web browser — everything else (Terraform, kubectl, Helm, etc.) gets installed **on the management EC2 server**, not your laptop, so your local machine needs almost nothing
- Roughly 2–3 hours for a first full run-through

---

6. Known Architecture Quirks & Things to Decide On

These were found by reading the actual `main.tf` and the actual Kubernetes manifest files directly, not carried over from the original walkthrough notes. Decide what you want to do about each one **before** you run `terraform apply` in Phase 3.

6.1 — Terraform provisions a real RDS instance the app never uses

`main.tf` contains:

```
resource "aws_db_subnet_group" "rds" { ... }

resource "aws_db_instance" "mysql" {
identifier = "medibot-mysql"
engine = "mysql"
instance_class = "db.t3.micro"
allocated_storage = 20
db_name = "medibotdb"
username = "admin"
password = var.db_password
...
}
```

...and `output.tf` even exports `rds_endpoint`. But the actual application manifests (`mysql.yaml`, in Phase 12) deploy MySQL as a **separate pod inside the cluster** instead, with its own `PersistentVolumeClaim` and its own `mysql-secret` — and the app's `MYSQL_HOST` environment variable points at the in-cluster `mysql-service`, never at the RDS endpoint. **The RDS instance is real, billed infrastructure that the application does not use at all.**

You have two reasonable options:

Option A — Remove it (recommended if you don't plan to migrate to RDS later). Delete these from `main.tf`:

- `resource "aws_security_group" "rds_sg"`
- `resource "aws_db_subnet_group" "rds"`
- `resource "aws_db_instance" "mysql"`

...and delete `output "rds_endpoint"` from `output.tf`, and the `variable "db_password"` block from `variable.tf` (nothing else references it once the DB instance is gone). Saves the `db.t3.micro` RDS cost entirely — a small amount, but it's paying for something `terraform apply` creates and nothing ever reads.

Option B — Keep it, deliberately, for a future migration. If you might later move MySQL off the cluster and onto a managed RDS instance (generally the more production-appropriate choice — it gets you automated backups, easier scaling, and one less stateful pod to operate), leave it as-is for now. Just know that right now, today, **it is dead weight** — nothing points at it.

This guide proceeds assuming you keep the current in-cluster MySQL setup (Option B's status quo) since that's what the real manifests deploy, but flags every place this matters.

6.2 — The sample app Deployment points at someone else's ECR image

`deployment.yaml`'s initial `image:` field (before Jenkins ever runs) is a placeholder pointing at a *different* AWS account's ECR repository, inherited from the project this was originally templated from. On the very first ArgoCD sync (before Jenkins has ever pushed a real image), the `medibot` pod will fail with `ImagePullBackOff` — **this is expected**, not a bug, and Phase 12 has you fix the `image:` field to point at your own ECR repo before you ever commit it, which avoids ever seeing that failure state.

6.3 — mysql-secret's password ships as a visible sample value

See the callout in Section 3 — change it before you commit, it's trivial to read back out of Git history otherwise.

Phase 1 — GitHub Repos + Token

Step 1 — Create GitHub Repos

Go to **github.com** → **New Repository** and create three **separate, private** repos:

Repo	Visibility	Initialize with README	Holds
`medibot-app`	Private	Yes	Application source code (Jenkinsfile, `pom.xml`, Java source)
`medibot-infra`	Private	Yes	Terraform (`main.tf`, `variable.tf`, `output.tf`)
`medibot-manifests`	Private	Yes	Kubernetes YAML only (what ArgoCD watches)

Why three repos, not one? This is the GitOps split described in Section 2, plus a third split for infrastructure-as-code: `medibot-app` holds application source and is owned by Jenkins (CI). `medibot-infra` holds the Terraform that provisions the AWS infrastructure itself and is only ever touched manually from the management server (Phase 3) — nothing in the CI/CD pipeline reads or writes to it. `medibot-manifests` holds only Kubernetes YAML and is owned by ArgoCD (CD). Keeping all three separate means a source-code change, an infrastructure change, and a "what's-currently-deployed" change are always distinguishable in Git history, and ArgoCD never needs to know anything about how the app is built or how the underlying AWS resources were created.

If skipped / merged into one repo: ArgoCD would end up watching your entire source tree, triggering sync attempts on every commit (including ones that have nothing to do with deployment), and you'd lose the clean separation between "this is what changed in the code" vs. "this is what's running in production."

Step 2 — Create a GitHub Personal Access Token

GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic) → Generate new token

Field	Value
Note	`medibot-token`
Expiration	90 days
Scopes	`repo` (all), `projects`

Click **Generate token** → **copy it immediately** (GitHub only shows it once).

> ***Where this secret goes:*** *this exact token gets pasted into* ***two*** *places later — Jenkins credential* `git` *(Step 33) and ArgoCD's repo connection (Step 44). Save it somewhere safe right now.*

Why: Jenkins needs this to clone your *private* repos and to push image-tag commits back into `medibot-manifests`. ArgoCD needs its own copy to poll `medibot-manifests` for changes.

If skipped: every single git operation in the pipeline fails immediately with an authentication error — nothing else in this guide will work without it.

Phase 2 — AWS Account Setup

Step 3 — Create an IAM User for Terraform (Console Access Only — No Access Keys)

AWS Console → IAM → Users → Create User

Field	Value
Username	`medibot-admin`
[x] Provide user access to the AWS Management Console	checked
Console password	Custom password (set one, save it)
Attach policy	`AdministratorAccess`

> ***Do NOT create an access key for this user.*** *Older versions of this guide had you download an access-key CSV and paste it into* `aws configure` *on the management server — long-lived static credentials sitting in plaintext on a box. Step 7 uses a much better approach (*`aws login --remote`*) that only needs this user to be able to* ***sign in to the Console with a password*** *— no access key is ever created or stored anywhere.*

Why: Terraform needs an identity with real AWS permissions to create every piece of infrastructure in this guide — VPC, EC2s, EKS, IAM roles. This user provides that identity; Step 7 is how the management server actually borrows it.

If skipped: `terraform apply` fails immediately with a "no valid credential sources" error before creating anything.

> `AdministratorAccess` is broad on purpose for a learning/personal project — for a real production account, scope this down to only the services Terraform actually touches.

Step 4 — Create an EC2 Key Pair

AWS Console → EC2 → Key Pairs → Create key pair

Field	Value
Name	`delete_later`
Type	RSA
Format	`.pem`

Click **Create** → **download** `delete_later.pem`, then lock down its permissions:

```
chmod 400 delete_later.pem
```

> ***Naming note:*** *the name* `delete_later` *isn't a style choice — it's exactly what* `main.tf`*'s* `key_name` *field is hardcoded to. Keep it exactly this. If you'd rather call it* `medibot-key`*, you must rename it in* ***both*** *the AWS key pair* ***and*** `main.tf`*'s* `key_name` *value — consistently, before running* `terraform apply`*.*

Why: Terraform references this key for all 4 EC2 instances it creates. It must already exist in AWS before `terraform apply` runs — Terraform does not create key pairs itself in this setup.

If skipped: `terraform apply` fails with "key pair does not exist."

Phase 3 — Management Server

Step 5 — Create the Management EC2 Instance

AWS Console → EC2 → Launch Instance

Field	Value
Name	`medibot-admin`
AMI	Ubuntu 22.04
Instance type	`t2.medium`
Storage	30 GB
Key pair	`delete_later`
Security group	Allow SSH (port 22) from your IP only

Launch, then connect:

```
ssh -i delete_later.pem ubuntu@<mgmt-public-ip>
```

Why a dedicated server instead of running Terraform from your laptop? Terraform's `apply`/`destroy` operations can take 10–15 minutes. If your laptop sleeps, your WiFi drops, or your SSH session closes mid-apply, you can end

up with a half-created, inconsistent `terraform.tfstate` that's painful to recover. Running it on a stable EC2 instance inside AWS removes that entire class of failure.

Step 6 — Install Tools on the Management Server

Run each block in order:

```
# --- AWS CLI ---
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt install unzip -y
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

```
# --- Terraform ---
sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install terraform
terraform version
```

```
# --- kubectl ---
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

```
# --- eksctl ---
curl -sLO "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname
-s)_amd64.tar.gz"
tar -xzf eksctl_$(uname -s)_amd64.tar.gz
sudo mv eksctl /usr/local/bin
eksctl version
```

```
# --- Helm ---
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version
```

```
# --- Git ---
sudo apt install git -y
```

Why each tool:

Tool	Role in this stack
AWS CLI	Talks to every AWS API directly (IAM, ECR, ACM, EC2, etc.)
Terraform	Declarative provisioning of the VPC, EC2s, EKS, IAM
kubectl	The standard Kubernetes command-line client, used to manage the EKS cluster directly

Tool	Role in this stack
eksctl	A higher-level EKS helper — used here specifically for IRSA setup (OIDC provider + IAM service accounts), which is tedious to do by hand
Helm	A package manager for Kubernetes — used to install the ALB Controller and the Prometheus/Grafana stack from pre-built "charts" instead of hand-writing hundreds of lines of YAML
Git	Needed to clone your Terraform repo (and later, manifests repos)

Step 7 — Connect the Management Server to AWS

> *This step deliberately does **not** use static access keys. Earlier drafts of this guide had you download an access-key CSV in Step 3 and paste it into `aws configure` — long-lived credentials sitting in plaintext on the box for as long as it exists. AWS CLI v2 ships a command built for exactly this situation instead: `aws login`, with a `--remote` mode specifically for "I'm SSH'd into a box with no local browser."*

Set the region once, then start the login flow:

```
aws configure set region ap-south-1
aws login --remote
```

This prints a URL and then waits:

```
Browser will not be automatically opened.
Please visit the following URL:

https://ap-south-1.signin.aws.amazon.com/v1/authorize?response_type=code&client_id=...

Enter the authorization code displayed in your browser:
```

What to do with that URL: copy it and open it in a browser **on your own laptop** — not on the EC2 instance, which has no GUI. That's the whole point of `--remote`: it skips trying to launch a local browser (which would fail over SSH) and instead gives you a link to open wherever you actually have one.

1. Open the URL on your own machine.
2. Log into the AWS Management Console with the `medibot-admin` user's **console password** from Step 3 (no access key needed at all — this flow authenticates the same way logging into console.aws.amazon.com in a browser normally does).
3. Approve the request. The page shows you an authorization code.
4. Copy that code back into the waiting terminal prompt on the management server and press Enter.

The CLI exchanges that code for **temporary credentials plus a refresh token**, cached locally (not written as a static key anywhere). As long as the refresh token stays valid, every subsequent AWS CLI call and every `terraform apply`/`destroy` on this box transparently uses auto-refreshing temporary credentials — you never type or paste a long-lived secret onto this machine at all.

Verify it actually worked:

```
aws sts get-caller-identity
```

You should get back something like:

```
{
"UserId": "AIDAXXXXXXXXXXXXXXXX",
"Account": "719152986284",
"Arn": "arn:aws:iam::719152986284:user/medibot-admin"
}
```

If `aws login --remote` fails with a region-related error, it's because the region wasn't set first — re-run the `aws configure set region ap-south-1` line above before retrying.

Why `medibot-admin` specifically, and not some other IAM identity? This is the Terraform-provisioning identity from Step 3, with broad `AdministratorAccess` — it's the only identity in this whole project that's allowed to create/destroy infrastructure itself. Every other credential you'll create later (Jenkins' ECR push user, for example) is deliberately narrower — and uses a plain access key instead, because it runs **unattended** inside a CI pipeline with no human present to approve a browser login each time, which is exactly the case `aws login` isn't designed for (its own docs call it out as "for local development").

> ***A production-grade EC2 instance*** *would typically skip human-driven login entirely and attach an* ***IAM Role*** *directly to the instance (an "instance profile") instead, so the box gets its own auto-rotating credentials with no login step of any kind. This guide uses* `aws login --remote` *instead because it needs a human to run Terraform interactively a handful of times, not continuously — for that workflow, a one-time browser approval beats either static keys or setting up a dedicated instance role.*

Step 8 — Clone the Terraform Repo and Deploy Infrastructure

```
git clone https://github.com/<YOUR-USERNAME>/medibot-infra.git
cd medibot-infra

terraform init # downloads providers, sets up local state
terraform plan # shows what WILL be created — review before applying
```

> *Remember Section 6.1: decide now whether you're keeping or removing the unused RDS resources, before you apply.*

If you kept the RDS resources (the default, unmodified `main.tf`), you must supply `db_password` — it has no default anymore (deliberately, so nobody ever commits a real password into `variable.tf` again):

```
terraform apply -var="db_password=<pick-your-own-password>"
```

If you removed the RDS resources per Option A, just run:

```
terraform apply -auto-approve
```

This takes roughly 15 minutes (EKS cluster creation is the slow part). When it finishes, capture the outputs:

```
terraform output
```

> ***Save these — but remember they're different every time you redeploy:*** - `eks_cluster_name` - `jenkins_public_ip` - `sonarqube_public_ip` - `nexus_public_ip` - `rds_endpoint` *(only if you kept the RDS resources — and remember, nothing actually uses it; see Section 6.1)*

Phase 4 — EKS Setup

Step 9 — Connect kubectl to the New EKS Cluster

```
aws eks --region ap-south-1 update-kubeconfig --name medibot-cluster

kubectl get nodes
# All 3 nodes should show STATUS = Ready
```

This command writes connection details (API server endpoint + an auth token helper) into `~/.kube/config` on the management server — that file is what every subsequent `kubectl` command reads to know *which* cluster to talk to, and it's also the exact file you'll copy onto the Jenkins server in Step 22.

Step 10 — Create an OIDC Provider for the Cluster

```
eksctl utils associate-iam-oidc-provider \
--region ap-south-1 \
--cluster medibot-cluster \
--approve
```

Why this matters (read this once, it explains a lot of what follows): Kubernetes pods normally have no AWS permissions at all — AWS has no idea what a "pod" even is. IRSA (IAM Roles for Service Accounts) bridges that gap: this command tells AWS "trust identity tokens issued by *this specific* EKS cluster." Once that trust exists, a pod can present its Kubernetes ServiceAccount token to AWS and *prove* "I am the `aws-load-balancer-controller` ServiceAccount in the `medibot-cluster`," and AWS will hand back real, temporary AWS credentials for whatever IAM Role is linked to that ServiceAccount — no static access keys stored anywhere in the cluster.

If skipped: the ALB Controller (Step 11) and anything else relying on IRSA has no way to get AWS permissions, and silently fails to create any AWS resources.

Step 11 — Install the AWS Load Balancer Controller

First, fetch the current VPC ID (it's different every redeploy):

```
aws ec2 describe-vpcs --filters "Name=tag:Name,Values=medibot-vpc" \
--query "Vpcs[0].VpcId" --output text
```

> *If a leftover CloudFormation stack exists from a previous attempt (`eksctl-medibot-cluster-addon-iamserviceaccount-...`), delete it first — see Teardown Step 6 for the exact disable-protection-then-delete commands.*

```
# 1. Download AWS's official IAM policy document for this controller
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
# 2. Create the IAM policy from it (just a rulesheet — attached to nothing yet)
aws iam create-policy \
--policy-name AWSLoadBalancerControllerIAMPolicy \
--policy-document file://iam_policy.json

# 3. Create a Kubernetes ServiceAccount AND link it to an IAM Role carrying that policy
eksctl create iamserviceaccount \
--cluster=medibot-cluster \
--namespace=kube-system \
--name=aws-load-balancer-controller \
--attach-policy-arn=arn:aws:iam::719152986284:policy/AWSLoadBalancerControllerIAMPolicy \
--approve \
--region ap-south-1

# 4. Install the actual controller pods via Helm, telling them to use that ServiceAccount
helm repo add eks https://aws.github.io/eks-charts
helm repo update

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
-n kube-system \
--set clusterName=medibot-cluster \
--set serviceAccount.create=false \
--set serviceAccount.name=aws-load-balancer-controller \
--set region=ap-south-1 \
--set vpcId=<VPC_ID from above>

# 5. Verify — wait for READY 2/2
kubectl get deployment -n kube-system aws-load-balancer-controller
```

What those 4 steps actually accomplish, in plain terms:

1. **OIDC provider** (Step 10) — "AWS, trust badges issued by my cluster."
2. **IAM Policy** — a list of allowed actions (create load balancer, etc.) — just a rulesheet, not attached to anyone yet.
3. `eksctl create iamserviceaccount` — does two things at once: creates the Kubernetes identity (ServiceAccount) *and* links it to an IAM Role carrying that policy. The badge now exists and is powerful, but nobody's wearing it yet.
4. **Helm install** — installs the actual controller pods and tells them to wear that badge (`serviceAccount.name=...`). Now the pods have real, working AWS permissions.

Why this component at all: it watches for Kubernetes `Ingress` objects and automatically provisions a matching AWS Application Load Balancer — this is what eventually makes MediBot reachable from the public internet.

If skipped: the app deploys fine inside the cluster but is completely unreachable from outside — no ALB ever gets created for the Ingress.

> ***cert-manager:*** *earlier versions of this guide installed cert-manager here. It's confirmed unused in this stack (TLS is handled entirely by an ACM certificate referenced directly in the Ingress annotation, not by cert-manager) —* ***skip it****.*

Step 12 — Install Metrics Server

```
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
# Verify — wait for READY 1/1
kubectl get deployment metrics-server -n kube-system
```

EKS node kubelets use self-signed certificates by default, which commonly makes metrics-server crash-loop. If that happens, patch it to skip TLS verification:

```
kubectl patch deployment metrics-server -n kube-system --type='json' \
-p='[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value":
"--kubelet-insecure-tls"}]'
```

Confirm metrics are flowing:

```
kubectl top nodes
kubectl top pods -n webapps
```

Why: the Horizontal Pod Autoscaler (`hpa.yaml`, created in Phase 12) needs the `metrics.k8s.io` API to read live CPU/memory usage and decide whether to scale replicas up or down. EKS does **not** install this by default — it's easy to forget.

If skipped: the HPA shows `Degraded` health in ArgoCD, with the error `unable to get metrics for resource cpu ... could not find the requested resource (get pods.metrics.k8s.io)` — it can never scale.

Step 13 — Verify the EBS CSI Driver (Already Installed by Terraform)

This one isn't something you install — Terraform already did it for you back in Step 8, as part of `terraform apply`. It's worth understanding and verifying now, because Phase 12's MySQL pod (and Phase 16's Prometheus/Grafana) both depend on it working, and a silent failure here shows up much later as a confusing "PVC stuck in Pending" with no obvious cause.

```
kubectl get pods -n kube-system | grep ebs-csi
# Expect to see ebs-csi-controller-* and ebs-csi-node-* pods, all Running

aws eks describe-addon --cluster-name medibot-cluster \
--addon-name aws-ebs-csi-driver --region ap-south-1 \
```

```
--query "addon.status"
# Expect: "ACTIVE"
```

What it does and why it's needed: a Kubernetes `PersistentVolumeClaim` by itself is just a *request* for storage — something still has to go talk to AWS and actually create an EBS volume, attach it to the right node, and format it. That "something" is the EBS CSI driver. `main.tf` sets this up completely on its own: an IAM Role for it via IRSA (`ebs_csi_role`), the matching `AmazonEBSCSIDriverPolicy` attachment, and the actual `aws_eks_addon.ebs_csi` resource — so by the time you reach this step, it should already be healthy. This step is purely a confidence check before you create your first PVC.

If it's broken: any `PersistentVolumeClaim` referencing the `ebs-sc` StorageClass (MySQL's in Phase 12, Prometheus/Grafana's in Phase 16) will sit in `Pending` forever, with `kubectl describe pvc <name>` showing something like `waiting for a volume to be created, either by external provisioner "ebs.csi.aws.com" or manually created by system administrator`.

Phase 5 — Bedrock IAM Permission

> *Moved up from where you might expect it (right after the app is deployed) to right here — the EKS node role this attaches to (`medibot-eks-node-role`) already exists at this point (Terraform created it in Phase 3), so doing it now means you'll never see the app's "AI service unavailable. Default triage applied." message, even on your very first deploy.*

Step 14 — Attach the Bedrock Policy to the EKS Node Role

```
aws iam create-policy \
--policy-name medibot-bedrock-policy \
--policy-document '{
"Version": "2012-10-17",
"Statement": [{
"Effect": "Allow",
"Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
"Resource": "*"
}]
}'

aws iam attach-role-policy \
--role-name medibot-eks-node-role \
--policy-arn arn:aws:iam::719152986284:policy/medibot-bedrock-policy
```

Why attach to the node role instead of a pod-level IRSA ServiceAccount (like Step 11 did for the ALB Controller)? This project takes the simpler path of granting the permission at the EC2 *node* level — every pod scheduled on that node inherits it through the instance's attached IAM role. It's less granular than IRSA (any pod on the node could technically call Bedrock), but it's significantly less setup for a project at this scale. A production-hardened version would scope this down with IRSA instead.

Bedrock model note: only Amazon Nova models are active in `ap-south-1`. You'll set this in the app's ConfigMap in Phase 12:

```
BEDROCK_MODEL_ID: "apac.amazon.nova-lite-v1:0"
BEDROCK_REGION: "ap-south-1"
```

To check which inference profiles are currently available:

```
aws bedrock list-inference-profiles --region ap-south-1 \
--query 'inferenceProfileSummaries[?contains(inferenceProfileId, `nova`)].inferenceProfileId' \
--output table
```

If skipped: every triage request in the app shows *"AI service unavailable. Default triage applied."* — there's no crash, just a silent fallback, which can be confusing to debug if you don't already know this is the cause.

Phase 6 — ECR Setup

Step 15 — Create the ECR Repository

```
aws ecr create-repository \
--repository-name medibot \
--region ap-south-1 \
--image-scanning-configuration scanOnPush=true
```

Save the `repositoryUri` from the output — it'll look like:
`719152986284.dkr.ecr.ap-south-1.amazonaws.com/medibot`

Step 16 — Create an IAM User Dedicated to Jenkins' ECR Push

AWS Console → IAM → Users → Create User

Field	Value
Username	`medibot-jenkins-ecr`
Attach policy	`AmazonEC2ContainerRegistryFullAccess`

Create access key → Application outside AWS → Download CSV. Save both values — this pair becomes the Jenkins `aws-creds` credential in Step 33.

Why a *separate* IAM user instead of reusing the Terraform one from Step 3? Least privilege — Jenkins only ever needs to push/pull Docker images, never to create or destroy infrastructure. If this credential ever leaked, the blast radius is limited to the ECR repo.

If skipped: Jenkins has no way to authenticate to AWS and the pipeline fails outright at the "Push to ECR" stage with an access-denied error.

Phase 7 — Jenkins Setup

Step 17 — SSH into the Jenkins EC2

```
ssh -i delete_later.pem ubuntu@<jenkins_public_ip>
```

Step 18 — Install Java + Jenkins

> **This is the single most common failure point in the whole build.** Installing only `openjdk-21-jre` reproduces a real bug we hit: Maven compiles using whichever `java` binary is on `PATH`, and MediBot's `pom.xml` targets Java 17 (`<java.version>17</java.version>`). A JRE-only Java 21 install has no `javac` at all, and even a full JDK 21 will fail to compile Java-17-targeted bytecode correctly in some configurations. **Install both**: 21 (what Jenkins itself requires to run) and 17 (what Maven needs to compile this app).

```
sudo apt update && sudo apt upgrade -y
sudo apt install fontconfig openjdk-21-jre openjdk-17-jdk -y

sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
sudo apt update
sudo apt install jenkins -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

> ***Do NOT** run `sudo update-alternatives --set java ...` to force the system-wide default to Java 17. Jenkins itself needs Java 21 minimum and will refuse to start if you do this (recoverable with `systemctl reset-failed jenkins && systemctl start jenkins`, but avoid it entirely). Instead, scope `JAVA_HOME` to Java 17 **only inside the Jenkinsfile's own pipeline environment**, which leaves Jenkins itself untouched:*
> ``groovy environment { JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64' PATH = "${JAVA_HOME}/bin:${env.PATH}" // ... rest of your environment vars } ``

Step 19 — Install Docker on the Jenkins EC2

```
curl https://get.docker.com | bash
sudo usermod -aG docker jenkins
sudo usermod -aG docker ubuntu
sudo systemctl restart docker
sudo systemctl restart jenkins
```

Why: the pipeline's `Docker Build` stage runs `docker build` directly on this host — Jenkins doesn't use a separate Docker daemon or Docker-in-Docker setup here, it shells out to the host's own Docker, which is why the `jenkins` system user needs to be in the `docker` group.

Step 20 — Install kubectl on the Jenkins EC2

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
kubectl version --client
```

Step 21 — Install Trivy on the Jenkins EC2

```
sudo apt-get install wget gnupg -y
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg --dearmor | sudo tee
/usr/share/keyrings/trivy.gpg > /dev/null
echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb
generic main" | sudo tee /etc/apt/sources.list.d/trivy.list
sudo apt-get update
sudo apt-get install trivy -y
trivy --version
```

Why: Trivy scans both the raw filesystem (`Trivy FS Scan` stage, catching vulnerable dependencies before you even build) and the final Docker image (`Trivy Image Scan` stage, catching vulnerable OS packages baked into the image) — basic supply-chain security hygiene.

Step 22 — Install AWS CLI on the Jenkins EC2

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo apt update
sudo apt install unzip -y
unzip awscliv2.zip
sudo ./aws/install
aws --version

# Create a symlink so the `jenkins` system user (whose PATH differs
# from your login shell's) can find the aws binary:
sudo ln -s /usr/local/aws-cli/v2/current/bin/aws /usr/bin/aws
sudo -u jenkins aws --version
```

Step 23 — Copy the kubeconfig to the Jenkins User

```
# On the management EC2:
cat ~/.kube/config
```

```
# On the Jenkins EC2:
sudo mkdir -p /var/lib/jenkins/.kube
sudo nano /var/lib/jenkins/.kube/config
# paste the kubeconfig content → Ctrl+X → Y → Enter
sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube
```

Why: this gives the Jenkins host its own working `kubectl` access to the cluster, independent of the management server (useful for manual debugging from Jenkins' shell — the pipeline itself doesn't actually deploy via kubectl, remember, that's ArgoCD's job).

Phase 8 — SonarQube Setup

Step 24 — Connect to the SonarQube EC2 via SSM

AWS Console → **EC2** → `medibot-sonarqube` → **Connect** → **Session Manager** → **Connect**

> *Note this instance uses **AWS Systems Manager (SSM)**, not an SSH key — no `.pem` file needed for this one.*

Step 25 — Install SonarQube (as a Docker Container)

```
sudo apt update
curl https://get.docker.com | bash
sudo usermod -aG docker ubuntu
newgrp docker
sudo systemctl start docker

docker run -d --name sonarqube \
--restart always \
-p 9000:9000 \
sonarqube:lts-community

docker logs sonarqube -f
# wait for: "SonarQube is operational"
```

Why a Docker container instead of a native install? SonarQube's official install docs assume a container runtime — it's the supported, low-maintenance path (no manual Java version juggling, no systemd unit files to author by hand), and `--restart always` means it survives an EC2 reboot automatically.

Step 26 — Configure SonarQube

Open `http://<sonarqube_public_ip>:9000`, log in with `admin`/`admin`, and change the password immediately when prompted.

Generate a token: top-right avatar → **My Account** → **Security**

Field	Value
Name	`jenkins-token`
Type	Global Analysis Token

Click **Generate** → **copy and save it immediately**.

Configure the webhook (this step is easy to skip and *will* bite you if you do):

Administration → **Configuration** → **Webhooks** → **Create**

Field	Value
Name	`jenkins`
URL	`http://<jenkins_public_ip>:8080/sonarqube-webhook/`

> The **trailing slash is required**. Without it, GitHub-style webhook URLs on Jenkins return a 404/405 and the call silently fails.

Why the token AND the webhook are two completely different things:

- **The token** is for *authentication* — proof that Jenkins is allowed to submit code for analysis at all. It's referenced by Jenkins credential ID `sonar-token`. Without it, the `SonarQube Analysis` stage fails outright with an auth error.
- **The webhook** solves a *timing* problem, not an authentication one. Jenkins submits code for analysis and moves on immediately — SonarQube's actual analysis happens asynchronously in the background, taking anywhere from a few seconds to a minute. The next pipeline stage, `Quality Gate`, needs to know "did it pass?" — but that answer isn't ready yet. The `Quality Gate` step *pauses and waits*. The webhook is literally how SonarQube reports back: once analysis finishes, SonarQube calls that URL to say "done, here's the result," which is what unblocks the waiting Jenkins stage.

If the webhook is missing: the `Quality Gate` stage hangs indefinitely (until its `timeout(time: 1, unit: 'HOURS')` eventually kills it) — you'll see `PENDING` forever in the SonarQube UI with no obvious link back to "the webhook is wrong."

Phase 9 — Nexus Setup

Step 27 — Connect to the Nexus EC2 via SSM

AWS Console → **EC2** → `medibot-nexus` → **Connect** → **Session Manager** → **Connect**

Step 28 — Install Nexus (as a Docker Container)

```
sudo apt update
curl https://get.docker.com | bash
sudo usermod -aG docker ubuntu
newgrp docker
sudo systemctl start docker

docker run -d --name nexus \
--restart always \
-p 8081:8081 \
sonatype/nexus3

docker logs nexus -f
# wait for: "Started Sonatype Nexus"
```

Step 29 — Configure Nexus

Open `http://<nexus_public_ip>:8081`. Get the auto-generated initial password:

```
docker exec nexus cat /nexus-data/admin.password
```

Log in → run the setup wizard → **set your own admin password and save it somewhere.**

Create two repositories (Settings → Repositories → Create Repository):

Format	Name	Version policy
maven2 (hosted)	`medibot-releases`	Release
maven2 (hosted)	`medibot-snapshots`	Snapshot

Create a dedicated Nexus user for Jenkins (Settings → Security → Users → Create):

Field	Value
Username	`jenkins`
Password	*pick one — save it*
Status	Active
Roles	`nx-admin`

> ***Critical:*** *whatever password you set here must be typed exactly the same into Jenkins'* `settings.xml` *in Step 35. A mismatch produces a 401 Unauthorized at the* `Publish to Nexus` *pipeline stage, with an error message that doesn't obviously point back to "these two passwords don't match."*

Step 30 — Update pom.xml with the Current Nexus URLs

In the `medibot-app` repo's `pom.xml`, inside `<distributionManagement>`, set both URLs to the **current** Nexus public IP:

```
<distributionManagement>
<repository>
<id>medibot-releases</id>
<url>http://<nexus_public_ip>:8081/repository/medibot-releases/</url>
</repository>
<snapshotRepository>
<id>medibot-snapshots</id>
<url>http://<nexus_public_ip>:8081/repository/medibot-snapshots/</url>
</snapshotRepository>
</distributionManagement>
```

Commit and push.

Why `pom.xml` only has URLs and no password: this file lives in your Git repo (even if private, it's still source-controlled text). The `<id>` tags (`medibot-releases`, `medibot-snapshots`) are just labels — Maven matches them at deploy time against a `<server>` entry with the *same* `<id>` in `settings.xml` (Step 35), which is where the actual username/password lives, safely outside of Git.

Phase 10 — Jenkins Configuration

Step 31 — Open Jenkins and Run Initial Setup

Open `http://<jenkins_public_ip>:8080`. Get the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Install suggested plugins, then create your own admin user.

Step 32 — Install Jenkins Plugins

Manage Jenkins → **Plugins** → **Available Plugins** — install all of these, then restart Jenkins:

Plugin	Purpose	Required?
Pipeline Stage View	Visual pipeline graph in the UI	Cosmetic only
SonarQube Scanner	Powers the `SonarQube Analysis` + `Quality Gate` stages	**Required**
Docker	Base Docker integration	**Required**
Docker Pipeline	Pipeline-friendly `docker.build()`-style syntax	**Required**
Maven Integration	Maven project/version support	**Required**
Pipeline Maven Integration	Provides the `withMaven {}` pipeline step	**Required**
Config File Provider	Injects the Maven `settings.xml` with Nexus credentials	**Required**
AWS Credentials	Provides `AmazonWebServicesCredentialsBinding` for the ECR push	**Required**
AWS Steps	AWS-specific pipeline steps	**Required**

Step 33 — Configure Tools

Manage Jenkins → **Tools**

Maven:

Field	Value
Name	`maven3`
Install automatically	[x] checked
Version	3.9.x or 3.10.x (any current release works)

SonarQube Scanner:

Field	Value
Name	`sonar-scanner`

Field	Value
Install automatically	[x] checked

Save.

Step 34 — Add All Credentials

Manage Jenkins → Credentials → System → Global → Add Credentials

Kind	ID	Value
Username with password	`git`	Username: your GitHub username · Password: the token from Step 2
Secret text	`sonar-token`	The SonarQube token from Step 26
AWS Credentials	`aws-creds`	Access Key ID / Secret Access Key from Step 16's CSV
Secret text	`aws-account-id`	`719152986284`

Step 35 — Configure the SonarQube Server in Jenkins

Manage Jenkins → System → SonarQube servers — check **"Environment variables"**, then add:

Field	Value
Name	`sonar`
Server URL	`http://<sonarqube_public_ip>:9000`
Authentication Token	`sonar-token` (select the credential, not a raw value)

Save.

Step 36 — Add Nexus Maven Settings

Manage Jenkins → Managed Files → Add a New Config

Field	Value
Type	Global Maven Settings XML
ID	`medibot-nexus`

Paste **only** the following — delete any default boilerplate content Jenkins pre-fills:

```
<?xml version="1.0" encoding="UTF-8"?>
<settings>
<servers>
<server>
<id>medibot-releases</id>
<username>jenkins</username>
<password>PASTE_SAME_PASSWORD_AS_STEP_29</password>
</server>
```

```
<server>
<id>medibot-snapshots</id>
<username>jenkins</username>
<password>PASTE_SAME_PASSWORD_AS_STEP_29</password>
</server>
</servers>
</settings>
```

> Nothing may exist after the closing `</settings>` tag — any extra content causes a "Non-parseable settings" error and the Maven deploy step fails.

Submit.

Phase 11 — Kubernetes Namespace

Step 37 — Create the Namespace

```
kubectl create namespace webapps
kubectl cluster-info
```

> ***Simplification vs. older versions of this guide:*** *a full RBAC setup (ServiceAccount + Role + RoleBinding + ClusterRole + ClusterRoleBinding + a long-lived Secret token for a "jenkins-token" credential) used to be documented here.* ***It's not actually needed*** *— this pipeline never has Jenkins talk to the Kubernetes API directly to deploy anything; ArgoCD handles 100% of cluster deployment via GitOps (see Section 2). A bare namespace is sufficient.*

Phase 12 — Review the Manifests Repo

Unlike `medibot-app` and `medibot-infra`, you do **not** create `medibot-manifests` from scratch — it's a **third, separate repo** that already exists with these 5 files already committed and already correct for this project (not placeholders — this is pulled directly from the real repo content). Without it, ArgoCD (Phase 13) would have nothing valid to sync, and the app would never get a database — which is exactly why earlier drafts of this guide missed this phase entirely: it's easy to forget a step exists when the repo it concerns is never touched during the AWS-side work.

```
git clone https://github.com/<YOUR-USERNAME>/medibot-manifests.git
cd medibot-manifests
ls
# LICENSE README.md deployment.yaml hpa.yaml ingress.yaml mysql.yaml storageclass.yaml
```

All five YAML files live at the repo **root** (ArgoCD's Application is configured with `Path: .` in Step 45 — it watches everything in the root). For a redeploy of this exact project, **none of these need to be edited** — walk through them below to understand what each one does and confirm nothing's missing, then move straight to Phase 13.

Step 38 — The EBS StorageClass

`storageclass.yaml` (as it already exists in the repo):

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: ebs-sc
provisioner: ebs.csi.aws.com
parameters:
type: gp3
fsType: ext4
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

Why `WaitForFirstConsumer`: this delays actually creating the EBS volume until a pod that needs it is actually scheduled onto a node — which matters because EBS volumes are zone-specific, and this ensures the volume gets created in the *same Availability Zone* as the pod that will use it, instead of possibly landing in the wrong zone and never being attachable.

Why `Retain`: if the PVC using this StorageClass is ever deleted, the underlying EBS volume is kept instead of being destroyed — a safety net against accidental data loss. (This is also exactly why Teardown Step 5 exists — these retained volumes don't clean themselves up and need a manual pass.)

Step 39 — The MySQL Manifest

`mysql.yaml` (as it already exists in the repo):

```yaml
---
apiVersion: v1
kind: Secret
metadata:
name: mysql-secret
namespace: webapps
type: Opaque
stringData:
MYSQL_ROOT_PASSWORD: "MediBot@2025"
MYSQL_DATABASE: "medibotdb"
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: mysql-pvc
namespace: webapps
spec:
accessModes: [ReadWriteOnce]
storageClassName: ebs-sc
resources:
requests:
storage: 5Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
```

```
name: mysql
namespace: webapps
spec:
selector:
matchLabels:
app: mysql
strategy:
type: Recreate
template:
metadata:
labels:
app: mysql
spec:
containers:
- name: mysql
image: mysql:8
env:
- name: MYSQL_ROOT_PASSWORD
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_ROOT_PASSWORD
- name: MYSQL_DATABASE
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_DATABASE
ports:
- containerPort: 3306
resources:
requests:
memory: "512Mi"
cpu: "500m"
limits:
memory: "1Gi"
cpu: "1"
volumeMounts:
- mountPath: /var/lib/mysql
name: mysql-data
volumes:
- name: mysql-data
persistentVolumeClaim:
claimName: mysql-pvc
---
apiVersion: v1
kind: Service
metadata:
name: mysql-service
namespace: webapps
spec:
ports:
- port: 3306
targetPort: 3306
selector:
app: mysql
```

> `MYSQL_ROOT_PASSWORD` is still the sample value here, committed in plaintext — see the callout in Section 3. It's not urgent to change for a redeploy of an already-running personal project, but if you ever intend to show this repo to anyone else or treat it as more than a learning exercise, rotate it and consider a proper secrets manager instead of a plain committed `Secret`. Note that `strategy: Recreate` (not the default `RollingUpdate`) matters here — MySQL's PVC is `ReadWriteOnce`, so two MySQL pods can never mount it simultaneously; a rolling update would try to start the new pod before killing the old one and get stuck.

Why `Deployment` instead of a `StatefulSet` for a database? A `StatefulSet` is the more "correct" Kubernetes primitive for stateful workloads like databases (stable network identity, ordered rollout/rollback) — this project uses a plain `Deployment` with `strategy: Recreate` instead, which works fine for a single-replica database but wouldn't scale cleanly to multiple MySQL replicas. Since this MySQL is never scaled beyond 1 replica, it's an acceptable simplification, just know it's not the textbook-correct choice for a production database.

Step 40 — The MediBot App Manifest

`deployment.yaml` (as it already exists in the repo):

```
---
apiVersion: v1
kind: ConfigMap
metadata:
name: medibot-config
namespace: webapps
data:
BEDROCK_REGION: "ap-south-1"
BEDROCK_MODEL_ID: "apac.amazon.nova-lite-v1:0"
MYSQL_HOST: "mysql-service"
MYSQL_DB: "medibotdb"
---
apiVersion: apps/v1
kind: Deployment
metadata:
name: medibot
namespace: webapps
spec:
replicas: 2
selector:
matchLabels:
app: medibot
template:
metadata:
labels:
app: medibot
spec:
containers:
- name: medibot
image: 719152986284.dkr.ecr.ap-south-1.amazonaws.com/medibot:v8
ports:
- containerPort: 8080
env:
- name: MYSQL_HOST
valueFrom:
configMapKeyRef:
```

```
name: medibot-config
key: MYSQL_HOST
- name: MYSQL_DB
valueFrom:
configMapKeyRef:
name: medibot-config
key: MYSQL_DB
- name: MYSQL_USER
value: "root"
- name: MYSQL_PASSWORD
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_ROOT_PASSWORD
- name: BEDROCK_REGION
valueFrom:
configMapKeyRef:
name: medibot-config
key: BEDROCK_REGION
- name: BEDROCK_MODEL_ID
valueFrom:
configMapKeyRef:
name: medibot-config
key: BEDROCK_MODEL_ID
resources:
requests:
memory: "256Mi"
cpu: "250m"
limits:
memory: "512Mi"
cpu: "500m"
livenessProbe:
httpGet:
path: /api/health
port: 8080
initialDelaySeconds: 90
periodSeconds: 10
readinessProbe:
httpGet:
path: /api/health
port: 8080
initialDelaySeconds: 90
periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
name: medibot-service
namespace: webapps
spec:
type: ClusterIP
ports:
- port: 80
targetPort: 8080
selector:
app: medibot
```

> *The `image:` tag (`:v8` above, or whatever it reads when you check) reflects whichever build number Jenkins last pushed — it doesn't matter what it says right now. Jenkins' `Update Manifests Repo` stage overwrites this exact line with the real build tag (`sed -i "s|image:.medibot:.|image: ...|g"`) on every successful pipeline run, so the very next build you trigger replaces it anyway. It already points at **your own** ECR account (`719152986284`), so there's no Section 6.2-style cross-account `ImagePullBackOff` to worry about on a redeploy — that concern only applies if you're starting over from the original tutorial template this project was based on, not from this repo as it exists today.*

Why the long `initialDelaySeconds: 90` on the health probes: Spring Boot applications with a database connection and AWS SDK initialization routinely take 60–90 seconds to become ready on `t3.medium`-class nodes. A shorter delay here would have Kubernetes repeatedly killing the pod for "failing" its liveness probe before it ever finished starting up — an easy trap to fall into.

Step 41 — The HPA Manifest

`hpa.yaml` (as it already exists in the repo):

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: medibot-hpa
namespace: webapps
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: medibot
minReplicas: 2
maxReplicas: 5
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70
```

This is what Phase 4's metrics-server feeds — without it installed and healthy, this HPA object exists but can never actually read current CPU usage, and shows `Degraded` in ArgoCD.

Step 42 — The Ingress Manifest

`ingress.yaml` (as it already exists in the repo):

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: medibot-ingress
namespace: webapps
annotations:
kubernetes.io/ingress.class: alb
```

```
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/certificate-arn:
    "arn:aws:acm:ap-south-1:719152986284:certificate/8653e0eb-a10d-4250-af1f-d6f0915ca1d5"
  rules:
  - host: medibot.nihitdevops.xyz
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: medibot-service
            port:
              number: 80
```

> ***This already has HTTPS fully configured*** *— the* `443` *listener,* `ssl-redirect`*, and the real* `certificate-arn` *are all already here, because the* `*.nihitdevops.xyz` *ACM certificate is the one thing this project deliberately keeps alive across every teardown (see Step 53). That's why, unlike a from-scratch deployment on a brand-new domain, this redeploy doesn't need an HTTP-only staging phase — the moment ArgoCD syncs this file and the ALB Controller provisions the ALB, it comes up with HTTPS already working. Phase 17 only needs to re-verify the certificate is still* `Issued` *and re-create the Route53 record that teardown deleted — it does* ***not*** *need to edit this file.*

Step 43 — Nothing to Commit (Unless You Changed Something)

Since all five files above already match what this project needs, a redeploy requires **no edits and no push** to `medibot-manifests` at all — just leave the clone from Step 38 as a local reference, and move straight to Phase 13 to point ArgoCD at the repo as it stands on GitHub.

If you did change something while reviewing (e.g. rotating `MYSQL_ROOT_PASSWORD` per the warning in Step 39), commit and push it now, before Phase 13, so ArgoCD's first sync picks up the change:

```
git add mysql.yaml # or whichever file(s) you actually edited
git commit -m "Rotate MySQL root password"
git push origin main
```

Phase 13 — ArgoCD Setup

Step 44 — Install ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl get pods -n argocd -w
```

```
# wait until all pods show Running
```

Expose the ArgoCD UI via a LoadBalancer (creates a new AWS NLB):

```
kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'

kubectl get svc argocd-server -n argocd
# copy the EXTERNAL-IP (a new NLB hostname every redeploy)
```

Get the auto-generated admin password:

```
kubectl get secret argocd-initial-admin-secret -n argocd \
-o jsonpath="{.data.password}" | base64 -d; echo
```

Step 45 — Configure ArgoCD

Open `http://<argocd-EXTERNAL-IP>`, log in with `admin` / the password above, then change it under **User Info** → **Update Password**.

Connect the manifests repo:

Settings → **Repositories** → **Connect Repo**

Field	Value
Method	HTTPS
URL	`https://github.com/<YOUR-USERNAME>/medibot-manifests.git`
Username	your GitHub username
Password	the GitHub token from Step 2

Create the Application:

Applications → **New App**

Field	Value
Name	`medibot`
Project	`default`
Sync Policy	Automatic → [x] Prune Resources, [x] Self Heal
Repo URL	`https://github.com/<YOUR-USERNAME>/medibot-manifests.git`
Revision	`main`
Path	`.` (manifests live at the repo root)
Cluster URL	`https://kubernetes.default.svc`

Field	Value
Namespace	`webapps`

Why the Cluster URL can be a generic-looking value: ArgoCD isn't a separate service hunting for your cluster somewhere out on AWS — it's running *as pods, inside your own EKS cluster* (you installed it there a moment ago). Every Kubernetes cluster automatically creates an internal Service called `kubernetes` in its `default` namespace, which always points to that cluster's own API server. `https://kubernetes.default.svc` is just the DNS name for "my own API server" — conceptually identical to how `localhost` always means "this machine," regardless of which machine you happen to be on.

What happens on first sync: ArgoCD reads every manifest already sitting in `medibot-manifests` (reviewed in Phase 12) and applies them all — `StorageClass`, `mysql-secret`, `mysql-pvc`, the `mysql` Deployment + Service, the `medibot-config` ConfigMap, the `medibot` Deployment + Service, the `HorizontalPodAutoscaler`, and the already-HTTPS-ready `Ingress` — in one pass.

What `Self Heal` actually does (this matters a lot for teardown later): if anyone or anything manually changes or deletes a resource ArgoCD manages, ArgoCD notices the drift from Git and reverts it — automatically, within seconds. This is powerful for production consistency, but it means you **cannot** manually `kubectl delete` an Ingress or Service that ArgoCD owns and expect it to stay deleted — see Teardown Step 1.

Phase 14 — Jenkins Pipeline Job

Step 46 — Create the Pipeline Job

Jenkins → **New Item** → **Name:** `medibot-ci` → **Pipeline** → **OK**

Field	Value
Build Triggers	[x] GitHub hook trigger for GITScm polling
Pipeline Definition	Pipeline script from SCM
SCM	Git
Repository URL	`https://github.com/<YOUR-USERNAME>/medibot-app.git`
Credentials	`git`
Branch	`*/main`
Script Path	`Jenkinsfile`

Save.

Step 47 — Set Up the GitHub Webhook

GitHub → `medibot-app` → **Settings** → **Webhooks** → **Add webhook**

Field	Value
Payload URL	`http://<jenkins_public_ip>:8080/github-webhook/`
Content type	`application/json`
Events	Just the push event
Active	[x] checked

> Same trailing-slash rule as the SonarQube webhook — GitHub will report delivery failures (HTTP 404/405) without it.

Step 48 — Run the Pipeline for the First Time

Jenkins → `medibot-ci` → **Build Now**

Watch each stage turn green in order:

```
Git Checkout → Compile → Test → Trivy FS Scan → SonarQube Analysis
→ Quality Gate → Build JAR → Publish to Nexus → Docker Build
→ Trivy Image Scan → Push to ECR → Update Manifests Repo
```

Once the last stage completes, ArgoCD detects the new commit in `medibot-manifests` within its polling interval (or immediately, if webhook-based sync is configured) and deploys the new image to EKS — the app goes live without Jenkins ever touching the cluster directly.

> *If the `medibot` pod was sitting in `ImagePullBackOff` before this first build completed (because the placeholder `:latest` tag from Step 40 doesn't exist in ECR yet), it should recover on its own within a minute or two of this stage finishing — no manual action needed.*

Phase 15 — Verify

Step 49 — Verify the Pods

```
kubectl get pods -n webapps
kubectl get svc -n webapps
kubectl get ingress -n webapps
kubectl logs -n webapps deployment/medibot --tail=50
```

Step 50 — Access the Application

```
http://<ALB-address> → MediBot UI
http://<ALB-address>/api/health → {"status":"UP"}
```

(Find `<ALB-address>` from `kubectl get ingress -n webapps`.)

Phase 16 — Monitoring

Step 51 — Install Prometheus + Grafana

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
cat > monitoring-values.yaml << 'EOF'
prometheus:
prometheusSpec:
storageSpec:
volumeClaimTemplate:
spec:
storageClassName: ebs-sc
accessModes: [ReadWriteOnce]
resources:
requests:
storage: 5Gi

grafana:
enabled: true
adminPassword: admin123
service:
type: LoadBalancer
persistence:
enabled: true
storageClassName: ebs-sc
size: 5Gi

nodeExporter:
enabled: true

kubeStateMetrics:
enabled: true
EOF

helm upgrade --install monitoring prometheus-community/kube-prometheus-stack \
-f monitoring-values.yaml \
-n monitoring \
--create-namespace

kubectl get pods -n monitoring -w
```

> *This relies on the same `ebs-sc` StorageClass and EBS CSI driver verified back in Step 13 — if that wasn't healthy, these PVCs will also sit in `Pending`.*

> *If the Grafana `LoadBalancer` stays `<pending>`, it's missing AWS annotations the controller needs to actually provision an NLB — patch it directly:* ``bash kubectl patch svc monitoring-grafana -n monitoring \ -p '{"metadata": {"annotations": {"service.beta.kubernetes.io/aws-load-balancer-type": "external", "service.beta.kubernetes.io/aws-load-balancer-scheme": "internet-facing"}}}' ``

Step 52 — Access Grafana

```
kubectl get svc -n monitoring | grep grafana
# copy EXTERNAL-IP

kubectl --namespace monitoring get secrets monitoring-grafana \
-o jsonpath="{.data.admin-password}" | base64 -d; echo
```

Open `http://<grafana-EXTERNAL-IP>`, log in with `admin` / the password above.

Import ready-made community dashboards (`+ → Import → ID → Import`):

ID	Dashboard
3119	Kubernetes cluster overview
1860	Node Exporter full
6417	Pod details

Or import a custom dashboard tailored to this exact stack (cluster overview, node resources, MediBot app metrics, MySQL, pod health, CI/CD infra health) via the Grafana HTTP API:

```
curl -X POST http://admin:<password>@<grafana-ip>/api/dashboards/db \
-H "Content-Type: application/json" \
-d @medibot-dashboard.json
```

Phase 17 — Domain + SSL Setup

Step 53 — ACM Certificate

The `*.nihitdevops.xyz` certificate is deliberately kept across teardowns — check it's still `Issued` before requesting a new one:

```
aws acm list-certificates --region ap-south-1 \
--query "CertificateSummaryList[?DomainName=='*.nihitdevops.xyz']"
```

If it's still there and `Issued`, **reuse that ARN directly** in Step

1 Only request a brand-new one if it's gone or expired:

AWS Console → Certificate Manager → Request certificate → Request a public certificate

Field	Value
Domain name	`*.nihitdevops.xyz`
Validation method	DNS validation

Add the resulting CNAME record in **Route53** → **nihitdevops.xyz** → **Create record**, then wait for ACM's status to flip to `Issued` and copy the Certificate ARN.

Step 54 — Confirm ingress.yaml Still Has the Right Certificate ARN

As shown back in Phase 12, Step 42, `ingress.yaml` in `medibot-manifests` already has HTTPS fully configured — the `443` listener, `ssl-redirect`, and a `certificate-arn` annotation. The only thing worth double-checking here is that the ARN in the file still matches the certificate you just verified in Step 53 (ACM certificate ARNs don't change across teardowns as long as you reuse the same certificate, so normally there is nothing to edit):

```
grep certificate-arn medibot-manifests/ingress.yaml
```

If it matches the ARN from Step 53, there's nothing to commit — move on to Step 55. If you ever *do* request a brand-new certificate (a different domain, or the old one expired), update the annotation with the new ARN, then commit and push — ArgoCD auto-syncs the change and the ALB Controller reconfigures the existing ALB listener.

Step 55 — Add the Route53 A Record

Route53 → **nihitdevops.xyz** → **Create record**

Field	Value
Record name	`medibot`
Record type	A
Alias	ON
Route traffic to	Alias to Application and Classic Load Balancer
Region	Asia Pacific (Mumbai)
Load balancer	the ALB address from `kubectl get ingress -n webapps`

Why an Alias record instead of a plain A record with a static IP? ALBs don't have a fixed IP address — AWS can change the underlying IPs at any time. An Alias record resolves dynamically to whatever the ALB's current IPs are, so it never goes stale the way a hardcoded IP would.

Step 56 — Test

Wait roughly 5 minutes for DNS propagation, then open:

```
https://medibot.nihitdevops.xyz
```

Post-Deployment Verification

Run this full sweep any time you want a health check of the whole stack:

```
kubectl get pods -n webapps
kubectl get svc -n webapps
kubectl get ingress -n webapps
kubectl get application -n argocd
kubectl get hpa -n webapps
kubectl logs -n webapps deployment/medibot --tail=50
kubectl logs -n webapps deployment/mysql --tail=20
kubectl get pods -n monitoring
kubectl get deployment -n kube-system aws-load-balancer-controller
kubectl get pods -n kube-system | grep ebs-csi
kubectl get nodes
kubectl top nodes
kubectl top pods -n webapps
```

Troubleshooting

Symptom	Fix
Pods `CrashLoopBackOff`	`kubectl logs -n webapps <pod-name> --previous`
`medibot` pod stuck `ImagePullBackOff` right after first ArgoCD sync	Expected if Jenkins hasn't run yet — see Section 6.2; resolves itself once Step 48 completes
MySQL connection refused	`kubectl exec -n webapps deployment/medibot -- env | grep MYSQL` — verify the connection string/credentials
AI triage shows "service unavailable"	Check `aws iam list-attached-role-policies --role-name medibot-eks-node-role` for the Bedrock policy; check `BEDROCK_MODEL_ID` is `apac.amazon.nova-lite-v1:0`
Jenkins build fails "release version 17 not supported"	`openjdk-17-jdk` is missing, or the Jenkinsfile's `JAVA_HOME` isn't scoped to it — see Step 18
Jenkins can't push to ECR	Verify `aws-creds` has `AmazonEC2ContainerRegistryFullAccess`; verify the `aws` binary symlink exists for the `jenkins` user
GitHub/SonarQube webhook returns 404/405	URL is missing its trailing slash — must end in `.../github-webhook/` or `.../sonarqube-webhook/`
SonarQube Quality Gate stuck at `PENDING`	Re-check the webhook URL and trailing slash in SonarQube's Administration → Webhooks
Maven `settings.xml` parse error	Ensure only the custom `<settings>` block exists in the Managed File — nothing after `</settings>`
ArgoCD sync failing	`kubectl logs -n argocd deployment/argocd-repo-server --tail=30`; confirm the Application's `Path` field is `.`, not `manifests`

Symptom	Fix
ArgoCD keeps recreating something you just deleted	`Self Heal` is on — delete the `Application` object itself first (`kubectl delete application medibot -n argocd`), *then* delete its managed resources
HPA shows `Degraded`	`metrics-server` isn't installed or is crash-looping — see Step 12
A PVC is stuck `Pending`	EBS CSI driver isn't healthy — see Step 13
Lost the `.pem` key file	Use EC2 Instance Connect to push a temporary key instead of recovering the original (valid ~60 seconds — push right before connecting): `aws ec2-instance-connect send-ssh-public-key --instance-id <id> --instance-os-user ubuntu --ssh-public-key file://temp_key.pub --availability-zone <az>`
`aws sts get-caller-identity` fails on the management server	The `aws login --remote` flow in Step 7 wasn't completed, or the refresh token expired — re-run `aws configure set region ap-south-1` then `aws login --remote` again

---

IP Address Dependencies

None of the 4 EC2 instances (Jenkins, SonarQube, Nexus, `medibot-admin`) have an Elastic IP by default. **Every stop/start assigns a brand-new public IP**, and several things are hardcoded to the old one — this is the single most common source of "it worked yesterday" confusion in this stack.

Instance	Referenced in	Consequence if stale
Jenkins	GitHub webhook Payload URL; SonarQube's "jenkins" webhook URL	Builds stop triggering; Quality Gate hangs
Nexus	`pom.xml` `<distributionManagement>` (both URLs)	`Publish to Nexus` stage fails with a connection error
SonarQube	Jenkins → Manage Jenkins → System → SonarQube servers → Server URL	`SonarQube Analysis` stage fails with a connection error
`medibot-admin`	Nowhere automated	You just need to look up the new IP before SSHing in again

Optional — Attach Elastic IPs to Avoid This Permanently

An Elastic IP (EIP) stays fixed across stop/start forever, at a trivial cost (~$0.005/hr while the instance is *stopped*, free while running). The trade-off: attaching one assigns a **new** address (you can't "freeze" the current dynamic one), so you update the 4 locations above **one more time** — but never again after that.

AWS Console → EC2 → Elastic IPs → Allocate Elastic IP address → Actions → Associate Elastic IP address → pick the instance. Repeat for all 4, then update the locations above.

Worth doing if you expect to repeatedly stop/start instances (e.g. pausing the whole stack overnight to save cost).

Complete Teardown Manual

> ***Do these in this exact order.*** *Doing them out of sequence is what causes orphaned load balancers, orphaned EBS volumes, and stuck Terraform destroys — this is the real, battle-tested sequence after hitting all three of those problems once.*

Step 1 — Delete the ArgoCD Application First

`Self Heal` silently recreates anything you delete manually in Steps 2–3 (a brand-new ALB/Ingress within seconds). Remove the `Application` object itself before touching anything it manages:

```
kubectl delete application medibot -n argocd
```

If it hangs, check for finalizers and patch them out:

```
kubectl get application medibot -n argocd -o yaml | grep finalizers
```

Step 2 — Delete Kubernetes-Managed Load Balancers Before the Cluster

The ALB (from the Ingress) and any NLBs (from Grafana/ArgoCD `LoadBalancer`-type Services) are AWS resources that Kubernetes created *outside* of Terraform's knowledge. Destroying the EKS cluster while these still exist orphans them permanently — they keep billing with nothing left able to delete them cleanly.

```
kubectl delete ingress medibot-ingress -n webapps
kubectl delete svc grafana -n monitoring --ignore-not-found
kubectl delete svc argocd-server -n argocd --ignore-not-found
```

Verify in **AWS Console** → **EC2** → **Load Balancers** that these are gone or draining (1–2 minutes) before moving on.

Step 3 — Let PVCs Release Cleanly (Avoid Orphaned EBS Volumes)

Prometheus, Grafana, and MySQL all use EBS-backed PersistentVolumeClaims. Deleting the cluster before these release cleanly leaves the underlying EBS volumes behind (and remember, `storageclass.yaml`'s `reclaimPolicy: Retain` means they're kept on purpose even after a clean PVC deletion — see Step 5 below).

```
kubectl delete namespace monitoring --ignore-not-found
kubectl delete namespace webapps --ignore-not-found
```

Wait for both namespaces to fully terminate (`kubectl get ns`) before Step 4.

Step 4 — Terraform Destroy

```
cd medibot-infra
terraform plan -destroy # sanity check — confirm the resource count
terraform destroy -auto-approve
```

If you still have the `db_password` variable (i.e. you kept the RDS resources per Section 6.1), supply it again here, the same way you did for `apply`:

```
terraform destroy -var="db_password=<the-same-value-you-used-to-apply>" -auto-approve
```

This tears down the 3 EC2 instances, the VPC + subnets + security groups, the EKS cluster + node group, the EBS CSI driver's IAM role, and (if present) the RDS instance.

Step 5 — Check for Leftover EBS Volumes

Because of `reclaimPolicy: Retain`, this step **always** finds something after a real deployment — it's not just a "just in case."

```
aws ec2 describe-volumes --filters Name=status,Values=available \
--query "Volumes[].{ID:VolumeId,Size:Size,Created:CreateTime}"

aws ec2 delete-volume --volume-id <volume-id> # for each orphan found
```

Step 6 — Delete the eksctl-Created CloudFormation Stack(s)

eksctl's ALB-controller IAM role stack ships with Termination Protection **on** by default — disable it first, then delete:

```
aws cloudformation update-termination-protection \
--stack-name eksctl-medibot-cluster-addon-iamserviceaccount-kube-system-aws-load-balancer-controller
\-no-enable-termination-protection --region ap-south-1

aws cloudformation delete-stack \
--stack-name eksctl-medibot-cluster-addon-iamserviceaccount-kube-system-aws-load-balancer-controller
\-region ap-south-1

aws cloudformation wait stack-delete-complete \
--stack-name eksctl-medibot-cluster-addon-iamserviceaccount-kube-system-aws-load-balancer-controller
\-region ap-south-1
```

Check **Console** → **CloudFormation** for any other `eksctl-medibot-*` stacks and repeat the same pattern.

Step 7 — Delete the IAM Policies (Only After Step 6)

`AWSLoadBalancerControllerIAMPolicy` is attached to the role Step 6's stack owns — deleting it first fails with `DeleteConflict`.

```
aws iam delete-policy --policy-arn \
arn:aws:iam::719152986284:policy/AWSLoadBalancerControllerIAMPolicy

aws iam delete-policy --policy-arn \
arn:aws:iam::719152986284:policy/medibot-bedrock-policy
```

Step 8 — Force-Delete the ECR Repository

```
aws ecr delete-repository --repository-name medibot \
--region ap-south-1 --force
```

Step 9 — Delete the IAM User and Its Access Key

```
aws iam list-access-keys --user-name medibot-jenkins-ecr
aws iam delete-access-key --user-name medibot-jenkins-ecr \
--access-key-id <key-id-from-above>
aws iam delete-user --user-name medibot-jenkins-ecr
```

Step 10 — Delete the EC2 Key Pair

```
aws ec2 delete-key-pair --key-name delete_later
```

Step 11 — Terminate the medibot-admin EC2

Not managed by Terraform (it was launched manually) — terminate it explicitly:

```
aws ec2 describe-instances --filters "Name=tag:Name,Values=medibot-admin" \
--query "Reservations[].Instances[].{ID:InstanceId,State:State.Name}"

aws ec2 terminate-instances --instance-ids <instance-id>
```

> *This also destroys the cached `aws login` credentials from Step 7 along with the instance — since they were already short-lived and auto-expiring rather than static keys, there's nothing further to revoke.*

Step 12 — Clean Up the Dangling Route53 Record

The ALB's A record has nothing left to point to once the ALB is gone:

AWS Console → Route53 → Hosted zones → nihitdevops.xyz → select the `medibot.nihitdevops.xyz` A record → Delete

> *Leave the hosted zone itself and the `*.nihitdevops.xyz` ACM certificate alone — reuse both next time (see Step 53).*

Step 13 — Final Verification Sweep

```
aws ec2 describe-instances --query \
"Reservations[].Instances[?State.Name!='terminated']" --region ap-south-1

aws cloudformation list-stacks --region ap-south-1 \
--stack-status-filter CREATE_COMPLETE UPDATE_COMPLETE
```

```
aws ecr describe-repositories --region ap-south-1

aws ec2 describe-volumes --filters Name=status,Values=available --region ap-south-1

aws rds describe-db-instances --region ap-south-1 \
--query "DBInstances[].DBInstanceIdentifier"
```

All five commands should come back empty (or show only things you deliberately intend to keep). If anything lingers, it's still billing you — go back and delete it.

PART II

Kubernetes Manifests Explained

A field-by-field walkthrough of every YAML file in medibot-manifests — what each resource does, and exactly how every object refers to another.

A field-by-field, beginner-friendly walkthrough of everything in medibot-manifests

> *This explains the 5 YAML files that live in the `medibot-manifests` GitHub repo — what every single field means, why it's there, and exactly how each resource refers to another (by name, by label, by DNS, or by raw AWS ARN). All YAML shown here is the **real, current content** of the actual repo, not a simplified example.*

---

1. Kubernetes Basics, in Plain English

Skip this section if you already know what a Pod, Deployment, and namespace are. If any of those words are new, read this first — every later section assumes you understand these.

Cluster — the whole system: a group of servers (called "nodes") that Kubernetes manages as one unit. Your EKS cluster has 3 nodes.

Node — one physical/virtual machine in the cluster (one of your 3 EC2 worker instances). Pods run *on* nodes.

Pod — the smallest thing Kubernetes actually runs. A Pod wraps one or more containers (almost always just one, in this project) and gives them a single shared network identity (one IP address). You never create a Pod directly in this project — a **Deployment** creates and manages Pods for you.

Deployment — a *manager* for Pods. You tell it "I want 2 copies of this container running, always" and it continuously makes that true: if a Pod crashes, the Deployment notices and starts a replacement; if you update the image, it replaces old Pods with new ones gradually. You almost never interact with Pods directly in day-to-day work — you edit the Deployment, and Kubernetes figures out which Pods to add, remove, or replace.

Service — a stable network address that sits in front of a changing set of Pods. Pods come and go and get new IPs constantly (every restart = a new IP) — a Service gives you one unchanging name and IP to talk to "whichever Pods are currently healthy," by continuously watching for Pods that match a **label selector** (see below).

Namespace — a folder-like boundary inside one cluster. Everything in this project lives in the `webapps` namespace, which is why every single `kubectl` command you'll ever run against these resources needs `-n webapps` (or `--namespace webapps`) — without it, `kubectl` defaults to the `default` namespace and will tell you the resource "not found," even though it genuinely exists, just somewhere else.

Labels — arbitrary key-value tags you stick on objects (e.g. `app: medibot`). They have no built-in meaning to Kubernetes by themselves — they only matter because *other* objects (Services, the Deployment's own Pod-management logic, the HPA) are configured to *search* for objects carrying a specific label. This "search by label" mechanism is how almost every cross-resource link in this project actually works under the hood, and it trips up nearly every beginner at least once — see the callouts throughout this doc.

Manifest — just the YAML file itself: a text description of "here's the resource(s) I want to exist." Kubernetes doesn't run the YAML file — it reads it, and makes the real cluster state match what's described. "Applying" a manifest (`kubectl apply -f file.yaml`, or in this project, **ArgoCD doing that automatically for you** every time you push to `medibot-manifests`) is the act of handing that description to Kubernetes.

The four things every manifest has:

```
apiVersion: ... # which version of the Kubernetes API defines this kind of object
kind: ... # what TYPE of object this is (Pod, Deployment, Service, Secret, ...)
metadata: # the object's name, namespace, and labels
name: ...
namespace: ...
spec: # the actual configuration — differs completely per kind
...
```

If you remember nothing else from this section: **Kubernetes resources refer to each other almost entirely by plain string matching** — a `name:` field in one object has to be typed *exactly* the same in another object's reference to it, or the link silently fails. There is no autocomplete, no compiler, no IDE catching a typo here — a single character difference is a real, common, hard-to-spot bug. This document calls out every single one of these links explicitly.

---

2. The Big Picture — 5 Files, 9 Resources

File	Resources inside (separated by `---`)	What it's for
`storageclass.yaml`	1 `StorageClass`	Tells Kubernetes *how* to create a real AWS EBS volume when something asks for storage
`mysql.yaml`	1 `Secret`, 1 `PersistentVolumeClaim`, 1 `Deployment`, 1 `Service`	The database — a MySQL container with durable disk storage
`deployment.yaml`	1 `ConfigMap`, 1 `Deployment`, 1 `Service`	The MediBot application itself
`hpa.yaml`	1 `HorizontalPodAutoscaler`	Automatically scales the MediBot app up/down based on CPU load
`ingress.yaml`	1 `Ingress`	Exposes the app to the public internet over HTTPS

That `---` you'll see inside the YAML files is a YAML "document separator" — it's how one file packs multiple independent Kubernetes objects together. Each file could technically be 3–4 separate files instead; they're just bundled together here because they're logically related (e.g. everything MySQL needs lives in one file).

All 9 objects live in the same namespace: `webapps`. All of them are synced into the cluster by ArgoCD automatically whenever `medibot-manifests`' `main` branch changes — you never run `kubectl apply` by hand against this repo's content.

---

3. storageclass.yaml Explained

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: ebs-sc
provisioner: ebs.csi.aws.com
```

```
parameters:
type: gp3
fsType: ext4
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

What even *is* a StorageClass? Think of it as a *template* or *recipe* for creating storage, not storage itself. When something (like MySQL, in the next section) says "I need 5GB of disk," it doesn't ask AWS directly — it creates a `PersistentVolumeClaim` (a *request*) that references this StorageClass by name, and the StorageClass's "recipe" is what actually gets followed to create a real EBS volume. The relationship is: **PersistentVolumeClaim (request) → StorageClass (recipe) → PersistentVolume (the real, created thing, auto-generated, you never write this one yourself) → real AWS EBS volume.**

Field	Meaning
`metadata.name: ebs-sc`	The name other objects reference. **[LINK]** `mysql.yaml`'s PVC references this exact string in its `storageClassName` field — if even one character differs, the PVC will never bind to any storage and sits in `Pending` forever.
`provisioner: ebs.csi.aws.com`	Which piece of software actually creates the volume. This string identifies the **AWS EBS CSI driver** — the same addon installed automatically by Terraform (`aws_eks_addon.ebs_csi` in `main.tf`) and verified in the main setup guide's Step 13. Without those pods running healthy in `kube-system`, this provisioner string points at nothing, and nothing ever gets created.
`parameters.type: gp3`	The EBS volume *type* to create — `gp3` is AWS's current general-purpose SSD type (cheaper and more configurable than the older `gp2`).
`parameters.fsType: ext4`	The filesystem to format the new volume with — a standard Linux filesystem.
`reclaimPolicy: Retain`	What happens to the real EBS volume if the PVC that's using it gets deleted. `Retain` means the volume survives, becoming "orphaned" (unattached to anything) rather than being destroyed. The alternative, `Delete`, would destroy the volume automatically along with the PVC. This is *why* the Teardown Manual in the main guide has a dedicated step to hunt down and manually delete leftover EBS volumes — they don't clean themselves up.
`volumeBindingMode: WaitForFirstConsumer`	*When* to actually create the real volume. `WaitForFirstConsumer` delays creation until a Pod that will use the volume is actually scheduled onto a specific node — this matters because EBS volumes exist in one specific Availability Zone, and this ensures the volume is created in the *same* AZ as the Pod, rather than risking a volume in the wrong AZ that the Pod's node can never attach to. The alternative, `Immediate`, creates the volume right away, before Kubernetes even knows which node will use it — riskier.

FAQ:

Q: Do I create this once, or every time I redeploy? A: It already exists as a committed file in `medibot-manifests` and gets re-applied by ArgoCD every time the cluster is rebuilt. You never hand-run this yourself — but note that a

`StorageClass` is a cluster-wide object (no `namespace:` field at all, unlike everything else in this project), so there's nothing to "re-create per namespace" either.

Q: What error do I see if this is missing or misspelled? A: Run `kubectl describe pvc mysql-pvc -n webapps` — you'll see something like `waiting for a volume to be created, either by external provisioner "ebs.csi.aws.com" or manually created by system administrator`, and the PVC's `STATUS` column (`kubectl get pvc -n webapps`) will show `Pending` indefinitely.

---

4. mysql.yaml Explained

This one file defines **4 separate Kubernetes objects**, separated by `---`. Go through them in order — each one builds on the last.

4.1 — The Secret

```yaml
apiVersion: v1
kind: Secret
metadata:
name: mysql-secret
namespace: webapps
type: Opaque
stringData:
MYSQL_ROOT_PASSWORD: "MediBot@2025"
MYSQL_DATABASE: "medibotdb"
```

Field	Meaning
`kind: Secret`	A Kubernetes object meant for sensitive values (passwords, tokens, keys) — mechanically almost identical to a `ConfigMap` (see Section 5.1), but Secrets are base64-encoded at rest and the Kubernetes dashboard/CLI hide their values by default in casual listing. **This is NOT real encryption** — anyone with permission to `kubectl get secret ... -o yaml` can trivially decode it (pipe the value through `base64 -d`). Treat it as "hidden from a casual glance," not "actually protected."
`type: Opaque`	The generic, no-special-meaning Secret type — used for arbitrary key/value data like this. (Other types exist for TLS certs, Docker registry credentials, etc., with extra built-in validation — not relevant here.)
`stringData:`	A convenience field: you write plaintext here, and Kubernetes automatically base64-encodes it into the real underlying `data:` field at the moment you apply it. If you ever `kubectl get secret mysql-secret -n webapps -o yaml`, you'll see `data:` with base64 strings, not `stringData:` with plaintext — that's expected.
`MYSQL_ROOT_PASSWORD` / `MYSQL_DATABASE`	Just key names you've chosen — they have no special meaning to Kubernetes itself. They only become meaningful because the MySQL container image (below) and the app's Deployment are both *written* to look for keys with these exact names.

Security note: this password (`MediBot@2025`) is committed in plaintext to Git. Anyone with read access to this repo — including its full history, even after you later change the value — can read it. Fine for a personal learning project; never acceptable for anything real without a proper secrets manager.

4.2 — The PersistentVolumeClaim

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: mysql-pvc
namespace: webapps
spec:
accessModes: [ReadWriteOnce]
storageClassName: ebs-sc
resources:
requests:
storage: 5Gi
```

Field	Meaning
`accessModes: [ReadWriteOnce]`	How many nodes can mount this volume read-write *at the same time*. `ReadWriteOnce` (the only mode EBS actually supports) means exactly **one** node, ever, at a time — this is *why* the MySQL Deployment below uses `strategy: Recreate` instead of the default rolling update: two Pods (old + new) can never both hold this volume simultaneously during a rollout.
`storageClassName: ebs-sc`	**[LINK]** This must exactly match `storageclass.yaml`'s `metadata.name`. This is the request-to-recipe connection described in Section 3.
`resources.requests.storage: 5Gi`	How large a volume to create — 5 gibibytes.

4.3 — The Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: mysql
namespace: webapps
spec:
selector:
matchLabels:
app: mysql
strategy:
type: Recreate
template:
metadata:
labels:
app: mysql
spec:
containers:
- name: mysql
image: mysql:8
```

```
env:
- name: MYSQL_ROOT_PASSWORD
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_ROOT_PASSWORD
- name: MYSQL_DATABASE
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_DATABASE
ports:
- containerPort: 3306
resources:
requests:
memory: "512Mi"
cpu: "500m"
limits:
memory: "1Gi"
cpu: "1"
volumeMounts:
- mountPath: /var/lib/mysql
name: mysql-data
volumes:
- name: mysql-data
persistentVolumeClaim:
claimName: mysql-pvc
```

> ***The most important, most commonly misunderstood thing in this entire file:*** `spec.selector.matchLabels` *(*`app: mysql`*)* ***must exactly match*** `spec.template.metadata.labels` *(*`app: mysql`*).* `selector` *is how the Deployment finds and "owns" the Pods it's responsible for managing;* `template` *is the actual recipe used to create those Pods. If these two don't match, Kubernetes* ***rejects the Deployment outright at apply time*** *with a validation error — it refuses to create a Deployment that can never find its own Pods. This is deliberate, by far the single most common beginner mistake when hand-writing a Deployment, and the reason* `kubectl apply` *sometimes fails with* `selector does not match template labels`*.*

Field	Meaning
`spec.selector.matchLabels`	The label the Deployment searches for to decide "which Pods belong to me."
`spec.strategy.type: Recreate`	On an update, kill the old Pod *completely* before starting the new one (instead of the default `RollingUpdate`, which starts the new Pod before killing the old). Required here because of the `ReadWriteOnce` volume constraint explained above.
`spec.template`	The actual Pod template — everything from here down describes what each Pod running MySQL looks like.
`image: mysql:8`	A public image from Docker Hub — MySQL version 8, officially maintained. No account/credentials needed to pull it (unlike the `medibot` app's own image, which comes from your private ECR).

Field	Meaning
`env:` with `valueFrom.secretKeyRef`	**[LINK]** Instead of a literal value, this says "go get the actual value from the Secret named `mysql-secret` (defined above, Section 4.1), specifically its `MYSQL_ROOT_PASSWORD` key." This is exactly how the MySQL container ends up knowing its own root password — the official `mysql:8` image is written to read this specific environment variable name at container startup and use it to set the actual database root password.
`ports.containerPort: 3306`	Documents which port the container listens on — MySQL's standard port. **Beginner note:** this field is informational/self-documenting; it does *not* by itself control or restrict any networking. Even without it, the Service below would still be able to route traffic to port 3306 on the Pod's IP — Kubernetes networking isn't firewalled by `containerPort` declarations.
`resources.requests` / `resources.limits`	`requests` is what Kubernetes *reserves* for scheduling purposes (guarantees this much is available on whichever node the Pod lands on) — it also affects in HPA math for autoscaling targets (see Section 6). `limits` is a hard ceiling: if the container tries to use more memory than its limit, it gets forcibly killed (`OOMKilled`); if it tries to use more CPU than its limit, it's simply throttled, not killed.
`volumeMounts` / `volumes`	**[LINK, two-step]** `volumeMounts.name: mysql-data` must match `volumes[].name: mysql-data` — this link is *local to this one Pod spec*, just connecting "where to mount it inside the container" (`mountPath`) to "which named volume to mount." Then separately, `volumes[].persistentVolumeClaim.claimName: mysql-pvc` must match the PVC's `metadata.name` from Section 4.2 — *that's* the link to the actual durable storage.

4.4 — The Service

```yaml
apiVersion: v1
kind: Service
metadata:
name: mysql-service
namespace: webapps
spec:
ports:
- port: 3306
targetPort: 3306
selector:
app: mysql
```

Field	Meaning
`metadata.name: mysql-service`	**[LINK]** This name is what `deployment.yaml`'s `MYSQL_HOST` ConfigMap value is set to (see Section 5.1) — Kubernetes' internal DNS automatically makes every Service resolvable by its name (within the same namespace) or its full form `mysql-service.webapps.svc.cluster.local` (from anywhere in the cluster).

Field	Meaning
(no `type:` specified)	Defaults to `ClusterIP` — meaning this Service only gets an internal cluster IP, reachable from *inside* the cluster, never from the public internet. That's intentional: nothing outside the cluster should ever talk to the database directly.
`ports.port: 3306`	The port *other things inside the cluster* connect to on this Service's own internal IP/DNS name.
`ports.targetPort: 3306`	The port on the *actual Pod* that traffic gets forwarded to. Here they're the same number, but they don't have to be (see `deployment.yaml`'s Service in Section 5.3 for an example where they genuinely differ).
`selector.app: mysql`	**[LINK]** This is the mechanism described in Section 1: the Service continuously watches for any Pod in this namespace carrying the label `app: mysql`, and automatically load-balances traffic across all of them — right now that's the one Pod the `mysql` Deployment manages.

FAQ:

Q: What happens to my data if I `kubectl delete pod mysql-xxxxx`? A: Nothing is lost. The Deployment notices its managed Pod is gone and immediately creates a replacement, which mounts the *same* PVC (`mysql-pvc`) and therefore the same underlying EBS volume — all the data is still there. Deleting a Pod that's managed by a Deployment is safe and routine; it's exactly how rolling updates and crash recovery work.

Q: What if I `kubectl delete pvc mysql-pvc`? A: This is destructive — Kubernetes detaches the real EBS volume from Kubernetes' tracking, and because `reclaimPolicy: Retain` is set, the underlying EBS volume itself survives (orphaned, costing money, doing nothing) rather than being deleted, but your database has no way to reattach to it automatically — a new PVC with the same name would get a *brand-new, empty* volume, not the old data.

Q: Why does the Deployment show only 1 replica, not 2 like the app? A: There's no `replicas:` field at all in `mysql.yaml`'s Deployment spec, and Kubernetes defaults to `1` when it's omitted. This is deliberate — a second MySQL replica wouldn't help without real MySQL replication configured, and would actively break given the `ReadWriteOnce` single-attachment volume constraint.

---

5. deployment.yaml Explained

Also 3 objects in one file: a `ConfigMap`, the app's `Deployment`, and its `Service`.

5.1 — The ConfigMap

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
name: medibot-config
namespace: webapps
data:
BEDROCK_REGION: "ap-south-1"
BEDROCK_MODEL_ID: "apac.amazon.nova-lite-v1:0"
MYSQL_HOST: "mysql-service"
```

```
MYSQL_DB: "medibotdb"
```

ConfigMap vs Secret — when to use which: mechanically almost identical (both are just key-value stores a Pod can read from), but a `ConfigMap` is for **non-sensitive** configuration — values that would be fine to see in plaintext in a `kubectl get configmap -o yaml`, in logs, or in a screenshot. Region names, model IDs, and service hostnames aren't secrets; passwords are — that's the entire reason this data is split across a `ConfigMap` here and a `Secret` in `mysql.yaml`.

Key	Where its value comes from	[LINK]
`BEDROCK_REGION`	Just a plain string, `ap-south-1`	None — self-contained
`BEDROCK_MODEL_ID`	Just a plain string, the Nova inference profile ID	None — self-contained, but must match an ID AWS Bedrock actually recognizes (see the main setup guide's Phase 5)
`MYSQL_HOST`	`"mysql-service"`	**This is the exact `metadata.name` of the Service defined in `mysql.yaml` Section 4.4.** This is how the app Pod knows *where* to find the database — not an IP address, a DNS name that Kubernetes' internal DNS resolves automatically.
`MYSQL_DB`	`"medibotdb"`	Matches the `MYSQL_DATABASE` value inside `mysql-secret` (Section 4.1) — same database name, kept in two different places because one is "what database name to create" (consumed by the MySQL container itself) and the other is "what database name to connect to" (consumed by the app). They're not programmatically linked to each other — they just need to be typed the same by whoever wrote this.

5.2 — The Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: medibot
namespace: webapps
spec:
replicas: 2
selector:
matchLabels:
app: medibot
template:
metadata:
labels:
app: medibot
spec:
containers:
- name: medibot
image: 719152986284.dkr.ecr.ap-south-1.amazonaws.com/medibot:v8
```

```
ports:
- containerPort: 8080
env:
- name: MYSQL_HOST
valueFrom:
configMapKeyRef:
name: medibot-config
key: MYSQL_HOST
- name: MYSQL_DB
valueFrom:
configMapKeyRef:
name: medibot-config
key: MYSQL_DB
- name: MYSQL_USER
value: "root"
- name: MYSQL_PASSWORD
valueFrom:
secretKeyRef:
name: mysql-secret
key: MYSQL_ROOT_PASSWORD
- name: BEDROCK_REGION
valueFrom:
configMapKeyRef:
name: medibot-config
key: BEDROCK_REGION
- name: BEDROCK_MODEL_ID
valueFrom:
configMapKeyRef:
name: medibot-config
key: BEDROCK_MODEL_ID
resources:
requests:
memory: "256Mi"
cpu: "250m"
limits:
memory: "512Mi"
cpu: "500m"
livenessProbe:
httpGet:
path: /api/health
port: 8080
initialDelaySeconds: 90
periodSeconds: 10
readinessProbe:
httpGet:
path: /api/health
port: 8080
initialDelaySeconds: 90
periodSeconds: 10
```

Field	Meaning
`replicas: 2`	Run 2 identical copies of this Pod at all times, for both redundancy (one can fail/restart without full downtime) and load distribution. **[LINK]** This number is the *starting point* — once the `HorizontalPodAutoscaler` in `hpa.yaml` is active and targeting this Deployment, the HPA takes over control of this field and adjusts it automatically based on load; manually editing `replicas:` here afterward has no lasting effect.
`selector`/`template.labels` `app: medibot`	Same matching-pair rule explained in Section 4.3 for MySQL — must be identical, or the Deployment is rejected outright.
`image: 719152986284.dkr.ecr.ap-south-1.amazonaws.com/medibot:v8`	Anatomy of this string: `719152986284` (your AWS account ID) `.dkr.ecr.` `ap-south-1` (region) `.amazonaws.com` (ECR's domain) `/medibot` (the repository name, created in the main guide's Phase 6) `:v8` (the specific image *tag* — this exact tag changes every time Jenkins successfully completes a build; whatever it currently says reflects the last build Jenkins ran).
`ports.containerPort: 8080`	**[LINK, triangle]** Must match the port Spring Boot's embedded web server actually listens on inside the container (8080 is Spring Boot's conventional default), **and** must match the port used in the `livenessProbe`/`readinessProbe` below, **and** must match the `targetPort` in this app's Service (Section 5.3) — if any one of these three numbers is wrong, the chain breaks (probes fail and Kubernetes kills the Pod repeatedly, or the Service can't reach a working port).
`env: MYSQL_HOST` / `MYSQL_DB` via `configMapKeyRef`	**[LINK]** Pulls from the `medibot-config` ConfigMap (Section 5.1) — `configMapKeyRef` works exactly like `secretKeyRef` (seen with MySQL above), just for non-sensitive data.
`env: MYSQL_USER value: "root"`	A **literal, hardcoded value** — not pulled from anywhere else. This is the one env var in this whole file that uses `value:` instead of `valueFrom:`, illustrating the two ways to set a container's environment variable: a plain string right here, or a reference to something else (ConfigMap/Secret).
`env: MYSQL_PASSWORD` via `secretKeyRef`	**[LINK, cross-file]** This is the single most important cross-file link in the whole project: this Deployment, defined in `deployment.yaml`, reaches into the `mysql-secret` Secret, which is defined in a **completely different file**, `mysql.yaml`. **Kubernetes doesn't care which YAML file originally created an object** — once applied, every object lives together in one shared pool inside its namespace, referenceable by name from anywhere else in that same namespace, regardless of which file "owns" it. Splitting resources across multiple files is purely an organizational convenience for humans reading the repo; it creates zero actual boundaries at runtime.
`env: BEDROCK_REGION` / `BEDROCK_MODEL_ID` via `configMapKeyRef`	Same pattern as `MYSQL_HOST`/`MYSQL_DB` — pulled from the ConfigMap.

Field	Meaning
`livenessProbe` vs `readinessProbe`	Two different questions Kubernetes asks the same HTTP endpoint (`GET /api/health` on port 8080), for two different purposes: **Liveness** = "is this container still working, or should I kill and restart it?" A failing liveness probe gets the Pod restarted. **Readiness** = "is this container ready to receive real user traffic right now?" A failing readiness probe doesn't restart anything — it just temporarily removes the Pod from the Service's pool of targets (Section 5.3), so users aren't routed to a Pod that's still starting up or temporarily overloaded, without killing it.
`initialDelaySeconds: 90`	Wait 90 seconds after the container starts before running the *first* probe at all. Spring Boot apps with a database connection and AWS SDK initialization commonly take 60–90 seconds to become genuinely ready — without this delay, Kubernetes would start probing (and likely killing, for liveness) the Pod before it ever had a chance to finish starting up.
`periodSeconds: 10`	After the initial delay, check again every 10 seconds, indefinitely.
`resources.requests.cpu: "250m"`	**[LINK]** `250m` means 250 "millicores" = a quarter of one CPU core. This exact value is what `hpa.yaml`'s `averageUtilization: 70` is calculated *against* — see Section 6.

5.3 — The Service

```yaml
apiVersion: v1
kind: Service
metadata:
name: medibot-service
namespace: webapps
spec:
type: ClusterIP
ports:
- port: 80
targetPort: 8080
selector:
app: medibot
```

Field	Meaning
`metadata.name: medibot-service`	**[LINK]** Referenced by name in `ingress.yaml`'s backend (Section 7) — this is how public internet traffic eventually finds its way to this Service.
`type: ClusterIP`	Explicit this time (MySQL's Service left it implicit) — internal-only, not reachable from outside the cluster by itself. Public access comes entirely through the separate `Ingress`/ALB layer, not through this Service's type.

Field	Meaning
`ports.port: 80` vs `ports.targetPort: 8080`	**A genuinely different pair of numbers** — the perfect concrete example of what `port` vs `targetPort` actually means: anything *inside the cluster* connecting to this Service uses port `80` (the Service's own, made-up, internal port number — you could pick almost anything here). Kubernetes then silently translates that to port `8080` on whichever Pod it routes the connection to — the port the actual container is listening on. The two numbers never have to match; they just happen to be identical over in `mysql-service` because `3306` made sense to use for both.
`selector.app: medibot`	**[LINK]** Finds Pods via the same label-matching mechanism as every other Service in this project — in this case, the 2 (or more, once the HPA scales up) Pods managed by the `medibot` Deployment.

---

6. hpa.yaml Explained

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: medibot-hpa
namespace: webapps
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: medibot
minReplicas: 2
maxReplicas: 5
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70
```

Field	Meaning
`kind: HorizontalPodAutoscaler` ("HPA")	A controller that automatically edits another object's `replicas` field based on live metrics — "horizontal" scaling means adding/removing whole Pod copies (as opposed to "vertical" scaling, which would mean giving an existing Pod more CPU/memory).
`spec.scaleTargetRef`	**[LINK]** A pointer, by `apiVersion` + `kind` + `name`, at the exact object this HPA controls. `name: medibot` must match `deployment.yaml`'s `metadata.name` exactly — this is how the HPA finds and starts managing that specific Deployment's `replicas` field.
`minReplicas: 2` / `maxReplicas: 5`	The HPA will never scale below 2 or above 5 Pods, no matter what the metrics say.

Field	Meaning
`metrics[0].resource.name: cpu`	Scale based on CPU usage (as opposed to memory, or a custom application metric — both are also possible with a more advanced HPA config, not used here).
`target.type: Utilization`, `averageUtilization: 70`	**[LINK]** "Keep the *average* CPU usage across all current Pods at around 70% of **what each Pod requested**" — and "what each Pod requested" is exactly `deployment.yaml`'s `resources.requests.cpu: "250m"` value from Section 5.2. If actual average usage climbs meaningfully above 70% of 250m, the HPA adds a Pod; if it drops well below, it removes one (down to the `minReplicas` floor).

This entire object depends on `metrics-server` being installed and healthy in the cluster (covered in the main setup guide's Phase 4, Step 12) — the HPA reads live CPU numbers from the `metrics.k8s.io` API, which only exists because `metrics-server` provides it. EKS does not install this by default.

FAQ:

Q: Why does `kubectl get hpa -n webapps` **show** `<unknown>/70%` **for the TARGETS column?** A: Almost always means `metrics-server` isn't running or isn't healthy — the HPA literally has no CPU numbers to compare against 70%. Can also happen if `deployment.yaml`'s Pods have no `resources.requests.cpu` set at all (there's nothing to calculate a percentage *of*) — not an issue here since it's explicitly set, but worth knowing as a general cause.

Q: Can I set `minReplicas: 0` **to save cost when there's no traffic?** A: Not with a plain HPA like this one — standard Kubernetes HPAs require at least 1 replica running at all times. True scale-to-zero needs an additional system like KEDA, which isn't part of this project.

Q: If I manually run `kubectl scale deployment medibot --replicas=4`**, what happens?** A: It'll work *momentarily*, but the HPA re-evaluates on its own interval (every 15 seconds by default) and will immediately overwrite your manual change back to whatever its own CPU-based calculation says — manual scaling and an active HPA fight each other, and the HPA always wins.

---

7. ingress.yaml Explained

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: medibot-ingress
namespace: webapps
annotations:
kubernetes.io/ingress.class: alb
alb.ingress.kubernetes.io/scheme: internet-facing
alb.ingress.kubernetes.io/target-type: ip
alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}, {"HTTPS": 443}]'
alb.ingress.kubernetes.io/ssl-redirect: "443"
alb.ingress.kubernetes.io/certificate-arn:
"arn:aws:acm:ap-south-1:719152986284:certificate/8653e0eb-a10d-4250-af1f-d6f0915ca1d5"
spec:
rules:
- host: medibot.nihitdevops.xyz
http:
```

```
paths:
- path: /
pathType: Prefix
backend:
service:
name: medibot-service
port:
number: 80
```

This is the only object in the entire project that reaches *outside* Kubernetes entirely — straight into raw AWS configuration — via a literal ARN string in an annotation.

What an `Ingress` is, conceptually: a *set of routing rules* for HTTP(S) traffic coming from outside the cluster — "requests for this hostname/path should go to that Service." On its own, an `Ingress` object does nothing; it needs an **Ingress Controller** watching for objects like this and actually acting on them. In this project, that controller is the **AWS Load Balancer Controller** (installed in the main guide's Phase 4) — a pod running in `kube-system` that watches for `Ingress` objects and provisions a real AWS Application Load Balancer to match.

Annotations, one at a time

Annotation	Meaning
`kubernetes.io/ingress.class: alb`	Tells Kubernetes *which* Ingress Controller should handle this object — a cluster could theoretically have multiple controllers installed (e.g. `alb` and `nginx`) watching for different Ingresses; this explicitly claims the AWS ALB Controller.
`alb.ingress.kubernetes.io/scheme: internet-facing`	The resulting ALB gets a public IP, reachable from the internet. The alternative, `internal`, would create an ALB only reachable from inside the VPC — not useful for a public-facing app.
`alb.ingress.kubernetes.io/target-type: ip`	Tells the ALB Controller to route traffic directly to individual Pod IP addresses (via the AWS VPC CNI networking this cluster uses), rather than routing to EC2 *instance* ports (`target-type: instance`). `ip` mode is the generally recommended approach — it works regardless of how `kube-proxy`/Services are configured internally, and is required if you ever add Fargate nodes.
`alb.ingress.kubernetes.io/listen-ports`	A literal JSON array (note it's a *string* containing JSON, not native YAML) telling the ALB which ports to listen on: plain HTTP on 80, and HTTPS on 443.
`alb.ingress.kubernetes.io/ssl-redirect: "443"`	Any request that arrives on port 80 (HTTP) gets automatically redirected to port 443 (HTTPS) instead of being served directly — this is what forces `http://medibot.nihitdevops.xyz` to become `https://medibot.nihitdevops.xyz` in a browser.
`alb.ingress.kubernetes.io/certificate-arn`	**[LINK, external to Kubernetes]** The exact ARN of a real certificate issued by AWS Certificate Manager (ACM) — covered in the main guide's Phase 17. This is how the ALB actually terminates HTTPS (decrypts incoming traffic) — the certificate itself lives entirely in AWS, outside Kubernetes' control; this annotation is just a pointer to it.

spec

Field	Meaning
`rules[0].host`	Only requests for this exact hostname (`medibot.nihitdevops.xyz`) match this rule — a single ALB can host many different domains with different rules, each pointing at a different Service, though this project only uses one.
`http.paths[0].path: /`	Matches all paths (everything after the host).
`pathType: Prefix`	`/` as a *prefix* match means literally every URL path matches (since every path starts with `/`) — `Prefix` matching is the most common choice. (The alternative, `Exact`, would only match the literal string `/` itself, excluding `/api/health` etc. — not what's wanted here.)
`backend.service.name: medibot-service`	**[LINK]** Must exactly match the Service's `metadata.name` from Section 5.3.
`backend.service.port.number: 80`	**[LINK]** Must match the Service's own `port` (not `targetPort`!) — the Ingress talks to the Service's port 80, and the Service *itself* is what translates that down to the Pod's port 8080.

The full request path, end to end

```
Browser → HTTPS:443 → ALB (decrypts TLS using the ACM cert)
→ Ingress rule matches host+path
→ medibot-service:80 (Service)
→ medibot Pod:8080 (container)
```

FAQ:

Q: Why is the `ADDRESS` column blank when I run `kubectl get ingress -n webapps`? A: The ALB Controller hasn't finished provisioning the real AWS ALB yet — this can take 1–2 minutes after the Ingress is first created. If it stays blank much longer, check the ALB Controller's own pod logs (`kubectl logs -n kube-system deployment/aws-load-balancer-controller`) for permission errors (IAM policy issues) or configuration errors (e.g. a malformed `listen-ports` JSON string).

Q: I edited `ingress.yaml` and pushed — nothing changed in the browser. Why? A: Three possible delays stack up: (1) ArgoCD needs to notice the Git change and sync (seconds, unless sync is paused), (2) the ALB Controller needs to notice the Kubernetes object changed and update the real ALB's listener rules (also usually fast), and (3) your browser or an intermediate DNS resolver may be caching the old response — try a hard refresh or `curl -v` to rule that out.

---

8. The Complete Reference Map

Every cross-reference in this project, in one place:

This object...	...references...	...by	Found in
`mysql-pvc` (PVC)	`ebs-sc` (StorageClass)	`storageClassName` field	`mysql.yaml` → `storageclass.yaml`
`mysql` (Deployment)	`mysql-secret` (Secret)	`secretKeyRef.name` (×2)	`mysql.yaml`, same file

This object...	...references...	...by	Found in
`mysql` (Deployment)	`mysql-pvc` (PVC)	`volumes[].persistentVolumeClaim.claimName`	`mysql.yaml`, same file
`mysql-service` (Service)	`mysql` Pods	`selector.app: mysql` (label match)	`mysql.yaml`, same file
`medibot-config` (ConfigMap) `MYSQL_HOST` value	`mysql-service` (Service)	plain string match (`"mysql-service"`) resolved via cluster DNS	`deployment.yaml` → `mysql.yaml`
`medibot` (Deployment)	`medibot-config` (ConfigMap)	`configMapKeyRef.name` (×4)	`deployment.yaml`, same file
`medibot` (Deployment)	`mysql-secret` (Secret)	`secretKeyRef.name`	`deployment.yaml` → `mysql.yaml` (cross-file!)
`medibot-service` (Service)	`medibot` Pods	`selector.app: medibot` (label match)	`deployment.yaml`, same file
`medibot-hpa` (HPA)	`medibot` (Deployment)	`scaleTargetRef.name`	`hpa.yaml` → `deployment.yaml`
`medibot-hpa` (HPA) CPU math	`medibot` (Deployment) `resources.requests.cpu`	implicit — percentage calculated against this value	`hpa.yaml` → `deployment.yaml`
`medibot-ingress` (Ingress)	`medibot-service` (Service)	`backend.service.name` + `.port.number`	`ingress.yaml` → `deployment.yaml`
`medibot-ingress` (Ingress)	AWS ACM certificate	raw ARN string in an annotation	`ingress.yaml` → outside Kubernetes entirely
`medibot-ingress` (Ingress)	AWS ALB Controller	`kubernetes.io/ingress.class: alb` annotation	`ingress.yaml` → a pod in `kube-system`
`storageclass.yaml`'s `ebs-sc`	AWS EBS CSI driver	`provisioner: ebs.csi.aws.com`	`storageclass.yaml` → an addon in `kube-system`

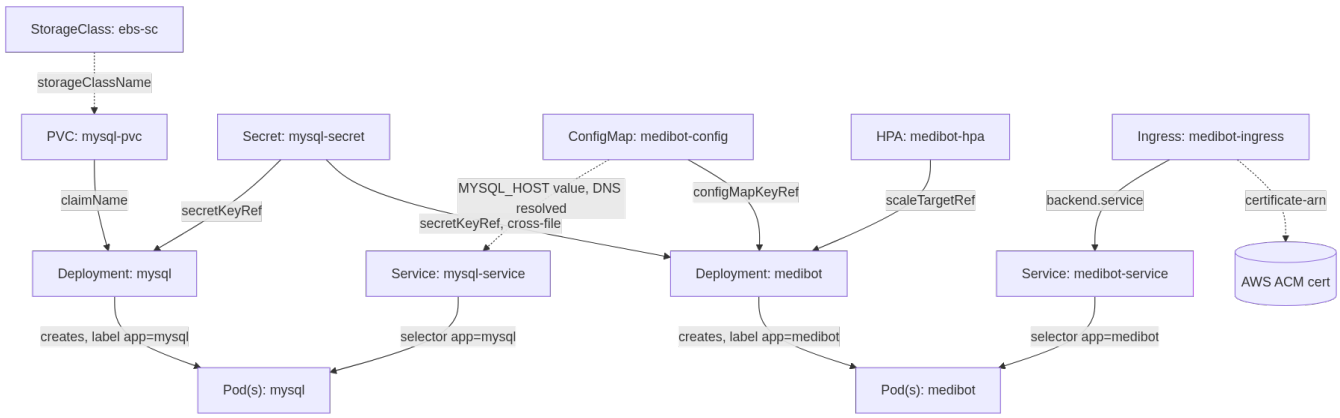


Figure 2.1 — Complete cross-resource reference map

9. General Beginner FAQ

Q: What's the difference between a Pod, a ReplicaSet, and a Deployment? A: A **Pod** is one running instance. A **ReplicaSet** keeps a specified number of identical Pods running (you never write one of these by hand in this project — Kubernetes creates it for you). A **Deployment** manages ReplicaSets on your behalf, specifically to support rolling updates — when you change the image or any Pod spec field, the Deployment creates a *new* ReplicaSet and gradually shifts traffic over, keeping the old one around briefly in case you need to roll back. You interact with the Deployment; it handles the rest.

Q: What is `kubectl` actually talking to? A: Your cluster's Kubernetes API server (the EKS-managed control plane) — every `kubectl` command is an HTTP API call, authenticated using the credentials in your `~/.kube/config` file (set up via `aws eks update-kubeconfig` in the main guide).

Q: Why does every command need `-n webapps`? A: Because every resource in this project was created *in* the `webapps` namespace, and `kubectl` defaults to the `default` namespace when you don't specify one — it's not that the resource doesn't exist, it's that you're looking in the wrong "folder."

Q: What do the different Pod statuses mean?

Status	Meaning
`Pending`	Scheduled, but not yet running — often waiting on an image pull, waiting for a PVC to bind, or waiting for a node with enough free resources
`ContainerCreating`	Image pulled, container is being set up
`Running`	Healthy and operating normally
`CrashLoopBackOff`	The container keeps starting and immediately exiting/crashing, and Kubernetes is backing off (waiting longer between each retry) rather than restart-looping as fast as possible
`ImagePullBackOff`	Kubernetes can't download the specified image — wrong tag, wrong account, or missing registry credentials
`OOMKilled` (seen in `kubectl describe pod`, not the status column)	The container exceeded its memory `limit` and was forcibly killed

Q: If I run `kubectl edit deployment medibot -n webapps` and change something, will it stick? A: It'll work immediately — but ArgoCD's `Self Heal` setting (covered in the main setup guide's Phase 13) notices the drift from what's committed in Git within seconds and reverts your manual change back to match the repo. In this GitOps setup, **Git is the only place a change is considered "real" and permanent** — manual `kubectl edit`/`patch` changes are temporary unless they also go into a commit.

Q: How do I see a resource's complete, real, live definition — not just what I wrote in the YAML file? A: `kubectl get <kind> <name> -n webapps -o yaml` — this shows you everything, including fields Kubernetes filled in automatically that you never wrote yourself (a `status:` section, auto-generated labels, resourceVersion, etc).

Q: What's the difference between `kubectl get` and `kubectl describe`? A: `get` gives you a short summary (good for "is it running, how many replicas"). `describe` gives you a much more detailed human-readable report, critically including recent **Events** at the bottom — this is almost always where you'll find the actual reason something is stuck or failing (a failed image pull, a scheduling failure, a failed probe). When something's wrong, reach for `describe` before `get -o yaml`.

Q: Labels vs. annotations — what's the real difference? A: Both are key-value metadata you attach to objects. **Labels** are meant to be *selected on* — Services, Deployments, and the HPA all use label matching to find the objects they care about (as seen throughout this doc). **Annotations** are not selectable at all — they're just arbitrary extra information attached to an object, often consumed by external tooling (like every `alb.ingress.kubernetes.io/...` annotation in `ingress.yaml`, which the ALB Controller reads but Kubernetes itself never queries or filters by).

Q: Could I run a second, independent copy of this whole app in a different namespace? A: Yes — namespaces are exactly the mechanism for that. You'd need to change every `namespace: webapps` to something else, and since Service DNS names include the namespace (`mysql-service.other-namespace.svc.cluster.local`), things would still resolve correctly as long as you're consistent.

---

10. Command Cheat Sheet

```
# See everything in the namespace at a glance
kubectl get all -n webapps

# Pods specifically, with more detail (node, restarts, age)
kubectl get pods -n webapps -o wide

# Full details + recent Events for one Pod (your first stop when debugging)
kubectl describe pod <pod-name> -n webapps

# Logs from a running Pod
kubectl logs -n webapps deployment/medibot --tail=100

# Logs from a Pod's PREVIOUS run (use when it's crash-looping)
kubectl logs -n webapps <pod-name> --previous

# Open a shell inside a running Pod
kubectl exec -it -n webapps deployment/medibot -- /bin/sh

# Check what env vars actually landed inside a container
kubectl exec -n webapps deployment/medibot -- env

# Decode a Secret's value
kubectl get secret mysql-secret -n webapps -o jsonpath="{.data.MYSQL_ROOT_PASSWORD}" | base64 -d;
echo
# Check the full live YAML, including auto-filled fields
kubectl get deployment medibot -n webapps -o yaml

# Check PVC binding status
kubectl get pvc -n webapps
kubectl describe pvc mysql-pvc -n webapps

# Check HPA's current view of CPU usage vs. target
kubectl get hpa -n webapps

# Check the Ingress and its assigned ALB address
kubectl get ingress -n webapps
```

```
# Watch a resource's status update live
kubectl get pods -n webapps -w
```

PART III

Jenkinsfile Explained

Every stage of the CI pipeline, in full detail — which credential it uses, which plugin makes it possible, and what happens if it fails.

A stage-by-stage, beginner-friendly walkthrough of the entire CI pipeline

> *This explains every single line of the real `Jenkinsfile` in the `medibot-app` repo — what each stage does, exactly which secret or credential it uses and where that credential was originally created, which Jenkins plugin makes each piece of syntax possible, and what breaks (and how) if any one piece is missing or misconfigured.*

---

1. What a Jenkinsfile Actually Is

A `Jenkinsfile` is a text file, committed to your source repo (`medibot-app`, at the root), written in a Jenkins-specific DSL ("Domain Specific Language") that's actually built on top of the **Groovy** programming language. You don't need to know Groovy to follow this project's Jenkinsfile — it uses the simpler **Declarative Pipeline** style (the `pipeline { ... }` wrapper), which looks and reads almost like structured configuration rather than a real program, even though Groovy is genuinely running underneath it.

Why keep this in the source repo instead of configuring stages by clicking through the Jenkins UI? This is "Pipeline as Code" — the entire build process is version-controlled, reviewable in a pull request, and identical no matter which Jenkins server runs it. The main setup guide's Phase 14, Step 46 configures the Jenkins job to read this exact file (`Script Path: Jenkinsfile`) directly from the repo on every run, instead of a human maintaining separate click-through configuration inside Jenkins itself.

How Jenkins actually finds and runs it, each time:

1. A push lands on `medibot-app`'s `main` branch.
2. GitHub's webhook (configured in Step 47) notifies Jenkins.
3. Jenkins' `medibot-ci` job (configured in Step 46) re-clones the repo.
4. Jenkins reads the `Jenkinsfile` at the repo root and parses the `pipeline { }` block.
5. It runs each `stage` inside, **strictly in the order they're written**, top to bottom — this file has no parallel stages, so each one fully finishes (or fails) before the next one starts.

---

2. The Full File, For Reference

```
pipeline {
agent any

tools {
maven 'maven3'
}

environment {
JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64'
PATH = "${JAVA_HOME}/bin:${env.PATH}"
SCANNER_HOME = tool 'sonar-scanner'
IMAGE_TAG = "v${BUILD_NUMBER}"
AWS_ACCOUNT_ID = credentials('aws-account-id')
```

```
AWS_REGION = 'ap-south-1'
ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com"
ECR_REPO = 'medibot'
IMAGE_FULL = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com/medibot"
}

stages {
stage('Git Checkout') {
steps {
git branch: 'main',
credentialsId: 'git',
url: 'https://github.com/nihitkumar78/medibot-app.git'
}
}

stage('Compile') {
steps {
sh 'mvn compile'
}
}

stage('Test') {
steps {
sh 'mvn test'
}
}

stage('Trivy FS Scan') {
steps {
sh 'trivy fs --format table -o trivy-fs-report.html .'
}
}

stage('SonarQube Analysis') {
steps {
withSonarQubeEnv('sonar') {
sh '''
$SCANNER_HOME/bin/sonar-scanner \
-Dsonar.projectKey=medibot \
-Dsonar.projectName=medibot \
-Dsonar.java.binaries=target
'''
}
}
}

stage('Quality Gate') {
steps {
timeout(time: 1, unit: 'HOURS') {
waitForQualityGate abortPipeline: false,
credentialsId: 'sonar-token'
}
}
}

stage('Build JAR') {
```

```
steps {
sh 'mvn package -DskipTests'
}
}

stage('Publish to Nexus') {
steps {
withMaven(globalMavenSettingsConfig: 'medibot-nexus',
maven: 'maven3',
traceability: true) {
sh 'mvn deploy -DskipTests'
}
}
}

stage('Docker Build') {
steps {
sh "docker build -t ${ECR_REPO}:${IMAGE_TAG} ."
}
}

stage('Trivy Image Scan') {
steps {
sh "trivy image --format table -o trivy-image-report.html ${ECR_REPO}:${IMAGE_TAG}"
}
}

stage('Push to ECR') {
steps {
withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
credentialsId: 'aws-creds']]) {
sh '''
aws ecr get-login-password --region ${AWS_REGION} | \
docker login --username AWS --password-stdin ${ECR_REGISTRY}

docker tag ${ECR_REPO}:${IMAGE_TAG} ${IMAGE_FULL}:${IMAGE_TAG}
docker push ${IMAGE_FULL}:${IMAGE_TAG}

docker tag ${ECR_REPO}:${IMAGE_TAG} ${IMAGE_FULL}:latest
docker push ${IMAGE_FULL}:latest
'''
}
}
}

stage('Update Manifests Repo') {
steps {
script {
cleanWs()
withCredentials([usernamePassword(
credentialsId: 'git',
usernameVariable: 'GIT_USER',
passwordVariable: 'GIT_PASS')]) {
sh '''
git clone https://${GIT_USER}:${GIT_PASS}@github.com/nihitkumar78/medibot-manifests.git
cd medibot-manifests
```

```
sed -i "s|image:.*medibot:.*|image: ${IMAGE_FULL}:${IMAGE_TAG}|g" deployment.yaml

git config user.name "Jenkins"
git config user.email "jenkins@medibot.com"
git add deployment.yaml
git commit -m "Update image to ${IMAGE_TAG} [build #${BUILD_NUMBER}]"
git push origin main
'''
}
}
}
}
}
}
```

3. The pipeline {} Block's Top-Level Keys

```
pipeline {
agent any

tools {
maven 'maven3'
}
...
}
```

Key	Meaning
`pipeline { }`	The outer wrapper that marks this whole file as a **Declarative Pipeline** (as opposed to the older, more free-form "Scripted Pipeline" style) — everything inside follows a fixed, predictable structure.
`agent any`	Tells Jenkins *where* to actually run this pipeline's steps — `any` means "run on whatever executor is available," which in this project's setup is the Jenkins EC2 instance itself (there's no separate pool of build agents configured — Jenkins is both the controller and the only worker). Every `sh` command in every stage literally runs as a shell command on this one EC2 box.
`tools { maven 'maven3' }`	**[LINK]** `'maven3'` must exactly match the name given to a Maven installation configured under **Manage Jenkins** → **Tools** (set up in the main setup guide's Phase 10, Step 33). This line tells Jenkins "before running any stage, make sure this named Maven installation is available, and add its `bin/` directory to `PATH` for the rest of this pipeline" — this is *why* plain `sh 'mvn compile'` works later without ever specifying a path to `mvn`. Requires the **Maven Integration** plugin.

4. The environment {} Block, Line by Line

```
environment {
JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64'
PATH = "${JAVA_HOME}/bin:${env.PATH}"
SCANNER_HOME = tool 'sonar-scanner'
IMAGE_TAG = "v${BUILD_NUMBER}"
AWS_ACCOUNT_ID = credentials('aws-account-id')
AWS_REGION = 'ap-south-1'
ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com"
ECR_REPO = 'medibot'
IMAGE_FULL = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com/medibot"
}
```

Everything defined here becomes a **shell environment variable** available in every single stage below, for the entire duration of this pipeline run. Think of this block as "set these once, up front, so I don't have to repeat them in every stage."

Line	Meaning
`JAVA_HOME = '/usr/lib/jvm/java-17-openjdk-amd64'`	A **single-quoted** Groovy string — a plain, literal value, no interpolation happens. This overrides, *just for this pipeline's own processes*, which Java installation gets used — specifically so Maven compiles with Java 17 (what this app's `pom.xml` requires) even though the Jenkins service itself runs on Java 21 system-wide. See Section 8 for exactly why this distinction matters, and the main setup guide's Phase 7, Step 18 for the full story of the bug this fixes.
`PATH = "${JAVA_HOME}/bin:${env.PATH}"`	A **double-quoted** Groovy string — this one *does* get interpolated by Groovy itself, before the shell ever runs anything. `${JAVA_HOME}` refers to the variable defined on the line directly above; `${env.PATH}` refers to the *existing* system PATH Jenkins already had. The result: this pipeline's `PATH` becomes "Java 17's `bin` folder, followed by everything that was already on PATH" — so when a later stage runs plain `java` or relies on Maven internally invoking `java`, it finds the Java 17 binary first.
`SCANNER_HOME = tool 'sonar-scanner'`	**[LINK]** `tool` is a built-in Jenkins pipeline function that looks up a named tool installation and returns its filesystem path. `'sonar-scanner'` must exactly match the name configured under **Manage Jenkins** → **Tools** → **SonarQube Scanner installations** (main guide Phase 10, Step 33). Requires the **SonarQube Scanner** plugin.
`IMAGE_TAG = "v${BUILD_NUMBER}"`	`BUILD_NUMBER` is a variable Jenkins provides automatically for every job — an integer that increments by 1 on every single run of this job, forever (1, 2, 3, ...). This line just prefixes it with `v`, giving tags like `v1`, `v2`, `v8`, etc. — this is *why* your ECR repo accumulates a growing list of versioned images over time.

Line	Meaning
`AWS_ACCOUNT_ID = credentials('aws-account-id')`	**[LINK, credential]** The `credentials()` helper function looks up a Jenkins Credential by its ID and injects its value as a plain environment variable — in this case a **Secret Text** credential, with ID `aws-account-id`, created in the main setup guide's Phase 10, Step 34, holding the literal string `719152986284`. Jenkins also automatically masks this value in console logs (replacing it with `****`) anywhere it would otherwise be printed — though for a non-sensitive value like an account ID, that masking is mostly a side-effect of using the same mechanism as real secrets, not a strict requirement.
`AWS_REGION = 'ap-south-1'`	A plain, hardcoded literal — no lookup, no interpolation, just a fixed string.
`ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com"`	Double-quoted, interpolated — builds the full ECR registry hostname by combining the account ID (from the credential above) with the fixed region.
`ECR_REPO = 'medibot'`	A plain, hardcoded literal — the name of the ECR repository created in the main setup guide's Phase 6.
`IMAGE_FULL = "${AWS_ACCOUNT_ID}.dkr.ecr.ap-south-1.amazonaws.com/medibot"`	The complete, pushable image reference: registry + repository, with no tag yet (the tag gets appended per-use later, e.g. `${IMAGE_FULL}:${IMAGE_TAG}`).

---

5. Every Stage, In Full Detail

Stage 1 — Git Checkout

```
stage('Git Checkout') {
steps {
git branch: 'main',
credentialsId: 'git',
url: 'https://github.com/nihitkumar78/medibot-app.git'
}
}
```

What it does: explicitly clones the `main` branch of `medibot-app` into Jenkins' workspace for this build. (Jenkins actually performs an *implicit* checkout automatically before any stage even runs, because the job itself is configured as "Pipeline script from SCM" — this explicit `git` step is a second, deliberate checkout matching the exact same repo, kept here for clarity/explicitness about exactly which repo and branch this pipeline builds from.)

Credential used: `git` (Username with Password kind) — created in the main setup guide's Phase 10, Step 34, holding your GitHub username and the Personal Access Token from Phase 1, Step 2. Required because `medibot-app` is a **private** repo — without valid credentials, this clone fails outright with an authentication error.

Plugin required: none beyond Jenkins' built-in Git support (the core **Git plugin**, which ships by default in any standard Jenkins install).

If this fails: the entire pipeline stops immediately — nothing downstream can run without the source code.

Stage 2 — Compile

```
stage('Compile') {
steps {
sh 'mvn compile'
}
}
```

What it does: runs `mvn compile` — Maven downloads all dependencies listed in `pom.xml` (Spring Boot, the AWS Bedrock SDK, MySQL connector, Jackson, etc.) and compiles all Java source files into `.class` files under `target/`. No tests run yet, nothing gets packaged yet — this stage exists purely to catch compilation errors as early as possible, before spending time on anything else.

Why this is where the Java 17 vs. 21 bug showed up: this is the *first* stage that actually invokes the `java`/`javac` toolchain — if `JAVA_HOME` weren't correctly scoped to Java 17 (Section 4), this is exactly where you'd see `error: release version 17 not supported`.

Credential used: none.

Plugin required: none beyond the Maven Integration plugin already covered by the `tools {}` block.

Stage 3 — Test

```
stage('Test') {
steps {
sh 'mvn test'
}
}
```

What it does: runs any unit tests defined under `src/test/java` using the `spring-boot-starter-test` dependency from `pom.xml`. If any test fails, Maven exits with a non-zero status, which fails this stage and stops the pipeline.

Credential used: none. **Plugin required:** none additional.

Stage 4 — Trivy FS Scan

```
stage('Trivy FS Scan') {
steps {
sh 'trivy fs --format table -o trivy-fs-report.html .'
}
}
```

What it does: runs Trivy (installed directly on the Jenkins EC2 in the main setup guide's Phase 7, Step 20 — **not** a Jenkins plugin, just a CLI binary invoked via `sh`) against the entire checked-out filesystem (`.`), scanning for known-vulnerable dependencies listed in `pom.xml` and any other flagged filesystem-level issues. Writes a human-readable HTML report (`trivy-fs-report.html`) into the workspace.

Note: this stage does **not** fail the build on findings by default (no `--exit-code` flag is set) — it's informational only in this configuration. A stricter setup would add `--exit-code 1` to actually block the pipeline on critical vulnerabilities.

Credential used: none. **Plugin required:** none — Trivy is a plain shell command, not a Jenkins integration.

Stage 5 — SonarQube Analysis

```
stage('SonarQube Analysis') {
steps {
withSonarQubeEnv('sonar') {
sh '''
$SCANNER_HOME/bin/sonar-scanner \
-Dsonar.projectKey=medibot \
-Dsonar.projectName=medibot \
-Dsonar.java.binaries=target
'''
}
}
}
```

What it does: submits the compiled code to SonarQube for static analysis (bugs, code smells, security hotspots, duplication, test coverage if configured).

`withSonarQubeEnv('sonar')`**:** this wraps the enclosed step with connection details for a SonarQube server — specifically, the one named `sonar` under **Manage Jenkins** → **System** → **SonarQube servers** (main setup guide Phase 10, Step 35), which holds the SonarQube server's URL and its authentication token credential. **[LINK]** The string `'sonar'` here must exactly match that configured server's name — this single name is what connects this stage to the right SonarQube instance and its stored token, without the token ever appearing directly in this file.

`$SCANNER_HOME/bin/sonar-scanner`**:** runs the actual scanner binary, whose location was resolved earlier by `tool 'sonar-scanner'` in the `environment {}` block.

`-Dsonar.projectKey` **/** `-Dsonar.projectName`**:** identifies this project inside SonarQube's own UI — must match (or SonarQube creates a new project entry) the project you're viewing results for.

`-Dsonar.java.binaries=target`**:** tells SonarQube's Java analyzer where to find the already-compiled `.class` files (from Stage 2) — the Java analyzer needs real bytecode, not just source, to run certain checks.

Credential used: indirectly, the SonarQube server's own stored authentication token (configured once, server-wide, in Step 35 — not a separate credential referenced per-build here).

Plugin required: SonarQube Scanner plugin (provides both the `withSonarQubeEnv` step and the `tool 'sonar-scanner'` installation type).

If this fails: usually an authentication problem with the SonarQube server connection, or the SonarQube server being unreachable (check the IP hasn't changed — see the main guide's IP Address Dependencies section).

Stage 6 — Quality Gate

```
stage('Quality Gate') {
steps {
timeout(time: 1, unit: 'HOURS') {
waitForQualityGate abortPipeline: false,
credentialsId: 'sonar-token'
}
}
}
```

What it does, and why it's a completely separate stage from the analysis itself: Stage 5 only *submits* code for analysis — SonarQube processes it asynchronously in the background, which can take anywhere from a few seconds to a minute or more. This stage *pauses the pipeline* and waits for SonarQube to report back a pass/fail verdict ("Quality Gate" status) for that analysis.

How the wait actually ends: not by polling — by a **webhook**. SonarQube, once it finishes analyzing, calls a webhook URL configured back in the main setup guide's Phase 8, Step 26 (`http://<jenkins_ip>:8080/sonarqube-webhook/`) to say "analysis done, here's the result." That webhook call is what unblocks this `waitForQualityGate` step. **Without that webhook configured correctly, this stage hangs for the full `timeout` duration (1 hour) and then fails** — a very common, very confusing failure mode, because the actual root cause (a missing/wrong webhook URL, or a missing trailing slash) is configured over in SonarQube's own UI, nowhere near this Jenkinsfile.

`abortPipeline: false`**:** even if the Quality Gate fails (code didn't meet SonarQube's quality thresholds), the pipeline is told to *continue anyway* rather than stop — meaning in this project's current configuration, a failing Quality Gate is informational only and never actually blocks a deployment. (Setting this to `true` would make a failing gate stop the whole pipeline.)

Credential used: `sonar-token` (Secret Text kind) — created in Phase 10, Step 34, holding the SonarQube analysis token generated back in Phase 8, Step 26.

Plugin required: SonarQube Scanner plugin (provides `waitForQualityGate`).

Stage 7 — Build JAR

```
stage('Build JAR') {
steps {
sh 'mvn package -DskipTests'
}
}
```

What it does: runs `mvn package`, which compiles (again, Maven is incremental but re-verifies), and this time actually produces the final build artifact — a runnable `.jar` file under `target/`, thanks to the `spring-boot-maven-plugin` declared in `pom.xml`'s `<build>` section (that plugin repackages a plain Spring JAR into an "executable" one with an embedded web server).

`-DskipTests`**:** deliberately skips re-running the test suite here — it already ran in Stage 3, so this avoids doing redundant work.

Credential used: none. **Plugin required:** none additional.

Stage 8 — Publish to Nexus

```
stage('Publish to Nexus') {
steps {
withMaven(globalMavenSettingsConfig: 'medibot-nexus',
maven: 'maven3',
traceability: true) {
sh 'mvn deploy -DskipTests'
}
}
}
```

What it does: runs `mvn deploy`, which — unlike `package` — doesn't just build the JAR locally, it **uploads** it to a remote Maven repository, specifically the Nexus repositories (`medibot-releases` / `medibot-snapshots`) declared in `pom.xml`'s `<distributionManagement>` section.

`withMaven(globalMavenSettingsConfig: 'medibot-nexus', ...)`**: [LINK]** this is the key piece — `mvn deploy` needs to *authenticate* to Nexus, and that authentication lives in a Maven `settings.xml` file, **not** in `pom.xml` (which only has the repository *URLs*, not credentials — see the manifests/pom.xml split explained in the main setup guide's Phase 9, Step 29). `globalMavenSettingsConfig: 'medibot-nexus'` tells Jenkins to inject the **Managed File** with that exact ID (created in Phase 10, Step 36) as the `settings.xml` used for this one Maven invocation — that file is where the Nexus username/password actually lives, matched up to `pom.xml`'s repository `<id>` tags.

`traceability: true`**:** adds extra console-log detail about exactly which configuration files Jenkins injected for this step — purely a debugging aid, doesn't change behavior.

Credential used: indirectly, the Nexus `jenkins` user's password, embedded inside the `medibot-nexus` Managed File (not a traditional Jenkins Credential object — Managed Files are a separate but related mechanism, provided by the same **Config File Provider** plugin).

Plugin required: Pipeline Maven Integration plugin (provides the `withMaven` step) and **Config File Provider** plugin (provides Managed Files like `medibot-nexus`).

If this fails with a 401: the password inside the `medibot-nexus` Managed File doesn't match the actual password set for the `jenkins` user in Nexus's own UI — these two have to be kept in sync manually, there's no automatic link between them.

Stage 9 — Docker Build

```
stage('Docker Build') {
steps {
sh "docker build -t ${ECR_REPO}:${IMAGE_TAG} ."
}
}
```

What it does: builds a Docker image from the `Dockerfile` at the repo root, tagging it locally as `medibot:v<build-number>` (both `ECR_REPO` and `IMAGE_TAG` come from the `environment {}` block, Section 4). This is a **double-quoted** `sh` string, so Groovy interpolates `${ECR_REPO}` and `${IMAGE_TAG}` itself, before the shell ever sees the command — see Section 8 for exactly why this matters.

Why this works without extra Docker setup: the Jenkins EC2 itself has Docker installed directly on it (main setup guide Phase 7, Step 19), and the `jenkins` system user was added to the `docker` group during that same step — `sh` here just runs a completely ordinary shell command against the host's own Docker daemon, no Docker-in-Docker or remote Docker API involved.

Credential used: none. **Plugin required: Docker** and **Docker Pipeline** plugins (technically the raw `sh "docker build..."` command would work even without these plugins installed, since it's just a shell call — but these plugins are what the main setup guide installs to support richer, alternative Docker syntax elsewhere in Jenkins' ecosystem; this project sticks to the simpler `sh` approach throughout).

Stage 10 — Trivy Image Scan

```
stage('Trivy Image Scan') {
steps {
sh "trivy image --format table -o trivy-image-report.html ${ECR_REPO}:${IMAGE_TAG}"
}
}
```

What it does: scans the Docker image *just built* (not the filesystem this time — the actual container image, including its base OS layer and all installed packages) for known vulnerabilities, writing a report to `trivy-image-report.html`. Same as Stage 4, this doesn't block the pipeline on findings in the current configuration.

Credential used: none. **Plugin required:** none — plain CLI call.

Stage 11 — Push to ECR

```
stage('Push to ECR') {
steps {
withCredentials([[$class: 'AmazonWebServicesCredentialsBinding',
credentialsId: 'aws-creds']]) {
sh '''
aws ecr get-login-password --region ${AWS_REGION} | \
docker login --username AWS --password-stdin ${ECR_REGISTRY}

docker tag ${ECR_REPO}:${IMAGE_TAG} ${IMAGE_FULL}:${IMAGE_TAG}
docker push ${IMAGE_FULL}:${IMAGE_TAG}

docker tag ${ECR_REPO}:${IMAGE_TAG} ${IMAGE_FULL}:latest
docker push ${IMAGE_FULL}:latest
'''
}
}
}
```

What it does, in order:

1. `aws ecr get-login-password` asks AWS for a short-lived (12-hour) authentication token for ECR, and pipes it directly into `docker login` — this is the AWS-recommended way to authenticate Docker to a private ECR registry, avoiding ever typing or storing a long-lived Docker password.

2 Tags the locally-built image (from Stage 9) with its full, pushable ECR path and version tag, then pushes it.

3 Repeats the same tag-and-push with `:latest` instead of the versioned tag — so the registry always has one "most recent" tag available, in addition to every historical version.

`withCredentials([[$class: 'AmazonWebServicesCredentialsBinding', credentialsId: 'aws-creds']])`**: [LINK]** this is a different mechanism from `credentials('aws-account-id')` used earlier in the `environment` block. Instead of injecting one plain string, `AmazonWebServicesCredentialsBinding` takes a stored **AWS Credentials**-kind Jenkins Credential (ID `aws-creds`, created in Phase 10, Step 34, holding the IAM access key pair from Phase 6, Step 16) and automatically sets the standard `AWS_ACCESS_KEY_ID`/`AWS_SECRET_ACCESS_KEY` environment variables for every command *inside this block only* — the AWS CLI then picks these up completely automatically, with no extra flags needed in the `sh` script itself.

Why this uses `'''` (triple single-quote) instead of a plain `sh "..."` like Stage 9: this is a multi-line script, and critically, it needs the **shell**, not Groovy, to expand `${AWS_REGION}`, `${ECR_REGISTRY}`, etc. — see Section 8 for the full explanation of why this distinction is deliberate and necessary here.

Credential used: `aws-creds` (AWS Credentials kind).

Plugin required: AWS Credentials plugin (provides the `AmazonWebServicesCredentialsBinding` class) and **AWS Steps** plugin.

If this fails with an access-denied error: verify the `aws-creds` IAM user still has `AmazonEC2ContainerRegistryFullAccess` attached, and that the symlink making `aws` visible to the `jenkins` system user (main guide Phase 7, Step 21) is still in place.

Stage 12 — Update Manifests Repo

```
stage('Update Manifests Repo') {
steps {
script {
cleanWs()
withCredentials([usernamePassword(
credentialsId: 'git',
usernameVariable: 'GIT_USER',
passwordVariable: 'GIT_PASS')]) {
sh '''
git clone https://${GIT_USER}:${GIT_PASS}@github.com/nihitkumar78/medibot-manifests.git
cd medibot-manifests

sed -i "s|image:.*medibot:.*|image: ${IMAGE_FULL}:${IMAGE_TAG}|g" deployment.yaml

git config user.name "Jenkins"
git config user.email "jenkins@medibot.com"
git add deployment.yaml
git commit -m "Update image to ${IMAGE_TAG} [build #${BUILD_NUMBER}]"
git push origin main
'''
}
}
}
}
```

This is the single most important stage in the entire file — it's the one and only moment this pipeline connects CI to CD. Everything before this point only built and validated an image; this stage is what actually tells the rest of the system "a new version exists."

`script { }`**:** declarative pipelines normally restrict you to a fixed set of step types inside `steps { }` — wrapping a block in `script { }` drops into full, unrestricted Groovy/Scripted Pipeline syntax for anything inside it. It's used here specifically because `cleanWs()` needs to run as a standalone statement before the `withCredentials` block even starts, which isn't expressible in the plain declarative style used by every other stage in this file.

`cleanWs()`**:** wipes the entire Jenkins workspace directory clean before continuing. **Why here specifically:** the workspace currently contains the `medibot-app` checkout from Stage 1, plus build artifacts from every stage since — none of that is relevant to this stage, which needs to clone a completely different repo (`medibot-manifests`) into a clean directory. Skipping this wouldn't necessarily break anything immediately, but risks stale files from a previous run interfering with a future one.

`withCredentials([usernamePassword(credentialsId: 'git', ...)])`**:** reuses the **same** `git` credential from Stage 1 — but note the *mechanism* is different: Stage 1 passed `credentialsId: 'git'` directly to the built-in `git` step, which handles authentication internally and never exposes the raw token. Here, `usernamePassword(...)` extracts the credential's username/password into two **named shell variables** (`GIT_USER`/`GIT_PASS`), because the following `sh` script needs to build a raw authenticated clone URL by hand (`https://${GIT_USER}:${GIT_PASS}@github.com/...`) — a pattern Git supports natively for HTTPS-based authentication.

`sed -i "s|image:.*medibot:.*|image: ${IMAGE_FULL}:${IMAGE_TAG}|g" deployment.yaml`**: [LINK]** the actual CI→CD handoff. `sed` is a text-stream editor; this specific command finds any line in `deployment.yaml` matching the pattern `image:` followed by anything, then `medibot:` followed by anything (i.e., the app container's `image:` line — see the Manifests Explained doc, Section 5.2), and replaces the *entire matched line* with the new, real image reference this exact build just pushed to ECR in Stage 11. This line is a **double-quoted** string (unlike the surrounding `sh '''...'''` block it sits inside), so `${IMAGE_FULL}` and `${IMAGE_TAG}` here are interpolated by **Groovy**, not the shell — see Section 8 for why this one line breaks the pattern of everything around it, deliberately.

`git add` **/** `commit` **/** `push`**:** commits that one-line change and pushes it straight to `medibot-manifests`' `main` branch, authenticated as a synthetic "Jenkins" git identity (`git config user.name`/ `user.email` — these aren't a real GitHub account, just metadata attached to the commit, visible in the repo's history).

What happens next, outside this Jenkinsfile entirely: ArgoCD, which is continuously polling this exact repo (main setup guide Phase 13), notices the new commit and automatically syncs the updated `deployment.yaml` into the cluster — Jenkins' job is done the moment this `git push` succeeds; it never touches Kubernetes directly.

Credential used: `git` (same credential as Stage 1, different consumption mechanism).

Plugin required: none beyond core Jenkins pipeline support — `cleanWs()` comes from the **Workspace Cleanup** plugin (commonly bundled/installed by default in most Jenkins setups), and `usernamePassword()` comes from core **Credentials Binding** plugin functionality.

If this fails: check the `git` credential's token hasn't expired (GitHub PATs created with an expiration, per the main setup guide's Phase 1, Step 2, stop working after 90 days) and that `medibot-manifests`' branch protection (if any) doesn't block direct pushes to `main`.

---

6. Master Credentials Table

Credential ID	Kind	Created in (main setup guide)	Holds	Used in this Jenkinsfile
`git`	Username with password	Phase 10, Step 34	GitHub username + Personal Access Token (from Phase 1, Step 2)	Stage 1 (`Git Checkout`, via `credentialsId:`); Stage 12 (`Update Manifests Repo`, via `usernamePassword()`)
`sonar-token`	Secret text	Phase 10, Step 34	SonarQube analysis token (from Phase 8, Step 26)	Stage 6 (`Quality Gate`, via `credentialsId:`)
`aws-creds`	AWS Credentials	Phase 10, Step 34	IAM access key pair for `medibot-jenkins-ecr` (from Phase 6, Step 16)	Stage 11 (`Push to ECR`, via `AmazonWebServicesCredentialsBinding`)
`aws-account-id`	Secret text	Phase 10, Step 34	Your plain AWS account ID, `719152986284`	`environment {}` block (via `credentials()`), feeding `ECR_REGISTRY`/`IMAGE_FULL` used in Stages 9, 10, 11, 12
`medibot-nexus` (a Managed File, not a Credential object)	Global Maven Settings XML	Phase 10, Step 36	Nexus `jenkins` user's password (from Phase 9, Step 29), embedded in a `settings.xml`	Stage 8 (`Publish to Nexus`, via `withMaven(globalMavenSettingsConfig:)`)
SonarQube server's own token	(configured server-wide, not per-credential here)	Phase 10, Step 35	Same `sonar-token` value, referenced by server name	Stage 5 (`SonarQube Analysis`, via `withSonarQubeEnv('sonar')`)

7. Master Plugin Table

Plugin	What it enables in this Jenkinsfile	What breaks without it
Maven Integration	The `tools { maven 'maven3' }` block, and the named Maven installation it refers to	`mvn` would not be on `PATH` — every `sh 'mvn ...'` call fails with "command not found"
SonarQube Scanner	`tool 'sonar-scanner'`, `withSonarQubeEnv('sonar')`, `waitForQualityGate`	Stages 5 and 6 fail outright — these functions don't exist without the plugin
Pipeline Maven Integration	The `withMaven(...)` step used in Stage 8	Stage 8 fails — `withMaven` is undefined
Config File Provider	Managed Files like `medibot-nexus`, referenced via `globalMavenSettingsConfig:`	Stage 8 fails — there's no `settings.xml` to inject, so `mvn deploy` has no Nexus credentials and fails with a 401

Plugin	What it enables in this Jenkinsfile	What breaks without it
AWS Credentials	The `AmazonWebServicesCredentialsBinding` class used in Stage 11	Stage 11 fails — `withCredentials` has no binding type for AWS keys
AWS Steps	General AWS-specific pipeline step support (installed alongside AWS Credentials in this project)	Not directly called by name in this file, but installed as part of the same AWS tooling setup
Docker / Docker Pipeline	Richer Docker-specific pipeline syntax elsewhere in Jenkins' ecosystem	This specific file's plain `sh "docker build..."` calls would actually still work without these, since they're just shell commands — these plugins matter more if you later adopt the `docker.build()`/`docker.withRegistry()` style syntax instead
Pipeline Stage View	Nothing this file depends on functionally — purely a visual pipeline graph in the Jenkins UI	Cosmetic only; the pipeline runs identically either way
Core Git support (bundled by default)	The `git branch:`, `credentialsId:`, `url:` step in Stage 1	Stage 1 fails — no way to check out source at all
Core Credentials Binding (bundled by default)	`credentials()` in the `environment` block, `usernamePassword()` in Stage 12	Both fail without this foundational plugin, which almost every Jenkins install has by default

---

8. The Groovy String Quoting Trap — Read This Once

This Jenkinsfile deliberately mixes **three different string styles**, and which one is used in each spot is not arbitrary — it controls *who* expands a `${...}` reference: **Groovy** (the pipeline script engine itself) or **the shell** (bash, running on the Jenkins EC2). Getting this backwards is one of the most common, hardest-to-diagnose mistakes when writing or editing a Jenkinsfile.

Style	Example from this file	Who expands `${...}`	When
Single-quoted Groovy string `'...'`	`sh 'mvn compile'`	Nobody — there's nothing to expand, it's a plain literal string passed as-is to the shell	Stages 2, 3, 4, 7
Double-quoted Groovy string `"..."`	`sh "docker build -t ${ECR_REPO}:${IMAGE_TAG} ."`	**Groovy**, immediately, before the shell ever sees the command	Stage 9, 10, and the `sed` line inside Stage 12
Triple single-quoted Groovy string `'''...'''`	`sh '''\n$SCANNER_HOME/bin/sonar-scanner ...\n'''`	**The shell**, when it runs — Groovy passes the text through completely untouched	Stages 5, 11, and the outer wrapper of Stage 12

Why it matters, concretely: inside Stage 11's `'''`-wrapped block, you'll see `${AWS_REGION}` and `${ECR_REGISTRY}`. Because this is a triple-*single*-quoted string, **Groovy does not touch these at all** — they're passed to bash completely literally, as the text `${AWS_REGION}`. Bash then sees that text, recognizes it as a shell variable reference, and expands it **itself** — successfully, because `AWS_REGION` and `ECR_REGISTRY` were already `export`-ed into the process environment by the `environment {}` block (Section 4), and every child process (including this `sh` script) inherits environment variables automatically. The end result looks identical either way in this

case (same values get substituted), but the *mechanism* is completely different depending on the quoting — and it would matter a great deal if, say, a value contained a `$` or backtick character Groovy would try to interpret differently than bash would.

The practical rule of thumb used consistently in this file: for a short, single-line command, a double-quoted `"..."` Groovy string is fine and common. For a longer, **multi-line** shell script (anything with multiple commands, pipes, or control flow), a triple-quoted `'''...'''` block is used instead — multi-line double-quoted Groovy strings are awkward and error-prone, and letting the shell itself handle variable expansion for a whole script block is simpler to reason about once you know which variables are actually `export`-ed into the environment (everything in this project's `environment {}` block is).

---

9. What Triggers This Pipeline, End to End

```
Developer runs `git push` on medibot-app's main branch
↓
GitHub sends an HTTP POST to http://<jenkins-ip>:8080/github-webhook/
(configured in the main setup guide's Phase 14, Step 47)
↓
Jenkins' "medibot-ci" job has "GitHub hook trigger for GITScm polling"
enabled (configured in Phase 14, Step 46) — this tells it to react
to that incoming webhook by polling the repo for changes right away
↓
Jenkins detects new commits on main, starts a new build
↓
Jenkins checks out the repo (implicit checkout from the job's SCM
config, then the explicit `git` step in Stage 1)
↓
Jenkins parses and runs this Jenkinsfile's `pipeline { }` block,
stage by stage, top to bottom, stopping immediately on the first
stage that fails (except where a stage is explicitly configured
to tolerate failure, like Quality Gate's abortPipeline: false)
```

A build can also be started manually at any time via **Jenkins UI** → `medibot-ci` → **Build Now** — this runs the exact same pipeline, just without a webhook having triggered it; useful for re-running after fixing a configuration issue without needing a new commit.

---

10. General Beginner FAQ

Q: What shell does `sh '...'` actually run? A: Whatever `/bin/sh` points to on the Jenkins EC2 — on this Ubuntu instance, that's typically `dash`, a lighter, more POSIX-strict shell than `bash`. Most simple commands behave identically either way, but some `bash`-specific syntax (certain advanced parameter expansions, `[[ ]]` test syntax) would not work inside a plain `sh` step.

Q: What's the difference between `withCredentials` and the plain `credentials()` function? A: `credentials('id')` (used in the `environment {}` block) is the simpler form — it injects one credential's value directly as an environment variable, available everywhere in the pipeline from that point on. `withCredentials([...])` (used in Stages 6, 11, 12) is more flexible — it can bind *multiple* parts of a credential at once (like splitting a username/password pair into two separate variables, as Stage 12 does), and critically, the

variables it creates **only exist inside that specific block** — once the block ends, they're gone, which is a slightly tighter security boundary than the `environment{}` approach.

Q: Does a failed stage automatically retry? A: No. A failed stage stops the entire pipeline immediately (unless explicitly wrapped in `catchError` or similar, which this file doesn't use anywhere). You have to manually trigger a new build — either by pushing a new commit, or clicking **Build Now** again.

Q: How does Jenkins know Stage 2 succeeded before starting Stage 3? A: Every `sh` step's exit code is checked automatically — a non-zero exit code (the standard Unix convention for "something failed") marks that step, and therefore that stage, as failed, which stops the pipeline. You never have to check exit codes yourself in a Jenkinsfile — the pipeline engine does it for every step implicitly.

Q: Where do I actually view the console output of a run? A: **Jenkins UI** → `medibot-ci` → **[build number]** → **Console Output** — this shows the complete, real-time (or historical) log of every command this pipeline ran, including the exact `sh` commands after Groovy's variable interpolation has already happened (so you'll see real values, not `${...}` placeholders, for anything Groovy-interpolated).

Q: What's `BUILD_NUMBER` vs. `BUILD_ID` vs. `BUILD_TAG`? A: All three are built-in Jenkins variables, automatically available in every pipeline without you declaring them. `BUILD_NUMBER` is the plain incrementing integer used in this file (`v1`, `v2`, ...). `BUILD_ID` is historically the same value as `BUILD_NUMBER` in modern Jenkins (it used to be a timestamp in very old versions). `BUILD_TAG` is a longer, pre-formatted string combining the job name and build number (e.g. `jenkins-medibot-ci-8`), handy as a unique identifier when you need one string instead of composing your own.

Q: Could I add a stage that deploys straight to Kubernetes with `kubectl apply`, instead of going through ArgoCD? A: Technically yes (Jenkins' kubeconfig was even copied onto this box back in the main guide's Phase 7, Step 23, partly for ad-hoc debugging access) — but doing so would break the GitOps model this entire project is built around: Git would no longer be a reliable record of what's actually deployed, since changes could land in the cluster without ever being committed anywhere. The whole reason this Jenkinsfile stops at "push an updated manifest to Git" (Stage 12) instead of "deploy directly" is to preserve that guarantee.